Certification of Imitation Controllers using Learned Lyapunov Functions

Master thesis by Josua Christoph Lindemann
Date: August 28, 2026, Date of submission: September 1, 2026

1. Review: Prof. Dr.-Ing. Rolf Findeisen
2. Review: Hendrik Alsmeier M.Sc.
Darmstadt

TECHNISCHE UNIVERSITÄT DARMSTADT

CCPS Control and Cyber-Physical Systems

Electrical Engineering and Information Technology Department

Certification of Imitation Controllers using Learned Lyapunov Functions

Master thesis by Josua Christoph Lindemann

1. Review: Prof. Dr.-Ing. Rolf Findeisen
2. Review: Hendrik Alsmeier M.Sc.

Date: August 28, 2026
Date of submission: September 1, 2026

Darmstadt

Technische Universität Darmstadt
Institut für Automatisierungstechnik und Mechatronik
Fachgebiet Control and Cyber-Physical Systems
Prof. Dr.-Ing. Rolf Findeisen

Thesis Statement pursuant to § 22 paragraph 7 of APB TU Darmstadt

I herewith formally declare that I, Josua Christoph Lindemann, have written the submitted thesis independently pursuant to § 22 paragraph 7 of APB TU Darmstadt without any outside support and using only the quoted literature and other sources. I did not use any outside support except for the quoted literature and other sources mentioned in the paper. I have clearly marked and separately listed in the text the literature used literally or in terms of content and all other sources I used for the preparation of this academic work. This also applies to sources or aids from the Internet. This thesis has not been handed in or published before in the same or similar form. I am aware, that in case of an attempt at deception based on plagiarism (§ 38 Abs. 2 APB), the thesis would be graded with 5,0 and counted as one failed examination attempt. The thesis may only be repeated once. For a thesis of the Department of Architecture, the submitted electronic version corresponds to the presented model and the submitted architectural plans.

Declaration on the Use of AI-Based Assistance

In accordance with the principles of academic integrity, the use of generative artificial intelligence and translation tools during the preparation of this thesis is disclosed as follows:

- **Employed Tools:** Google Gemini (3.1 Pro and 3.7 Flash) and DeepL (DeepL SE).
- **Scope of Application:** These tools served as aids for refining language, formulating text, and checking grammar, as well as for assisting with code implementation, such as code structuring, syntax support, and debugging.
- **Intellectual Contribution:** All underlying concepts, system architectures, theoretical derivations, research decisions, and final code validations represent the author's own intellectual work. The author assumes full responsibility for the correctness and scientific integrity of all text, data, and software implementations.

Darmstadt, _____ _____

Date Josua Christoph Lindemann

Abstract

Contents

List of Variables and Symbols

Symbol	Description
System Dynamics & Control	
n_x	State space dimension
n_u	Control input space dimension
$\mathcal{X}$	Admissible state set
$\mathcal{U}$	Admissible input set
$\underline{x}$	Lower state bound
$\overline{x}$	Upper state bound
$\underline{u}$	Lower control input bound
$\overline{u}$	Upper control input bound
T_s	Sampling time
k	Discrete time step index
$x(t)$	Continuous-time state vector
x_k	Discrete-time state vector
$u(t)$	Continuous-time control input vector
u_k	Discrete-time control input vector
$\tilde{x}$	State deviation from equilibrium
$\tilde{u}$	Input deviation from equilibrium
$x^\star$	System equilibrium point
$u^\star$	Input reference at $x^\star$
x_{k+1}	Discrete-time successor state
x^+	One-step successor state
$\pi(x)$	State-feedback policy
$V(x)$	Positive definite Lyapunov function
$f_c(x,u)$	Continuous-time system dynamics
$f(x_k,u_k)$	Discrete-time system dynamics
Optimal Control & Model Predictive Control (MPC)	
N	Prediction horizon
$J(x_k,(u_{i\|k})_{i=0}^{N-1})$	Finite-horizon MPC cost objective function
$\ell(x,u)$	Stage cost function
$V_f(x)$	Terminal cost function
$\mathcal{X}_f$	Terminal constraint set
Q	State weighting matrix
R	Input weighting matrix
$\pi_0(x)$	Closed-loop MPC feedback policy

Symbol	Description
$\pi_f(x)$	Auxiliary local terminal control law
$u^\star_{0\|k}$	Optimal control input at time step k
$V_N(x)$	Optimal value function
$x_{i\|k}, u_{i\|k}$	Predicted state and input at horizon step i initialized at step k
$(\tilde{u}_{i\|k+1})$	Shifted candidate input sequence
$(\tilde{x}_{i\|k+1})$	Shifted candidate state sequence
A	Linearized system state matrix
B	Linearized system input matrix
K	LQR feedback gain matrix
P	DARE solution matrix
$\mathcal{X}_N$	Feasible initial state set
Machine Learning	
L	Total layer count
l	Layer index
n_l	Layer l neuron count
n_0	Input layer dimension
n_L	Output layer dimension
n_h	Number of hidden neurons
$a^{(0)}$	Network input vector
$a^{(l)}$	Layer activation vector
$z^{(l)}$	Layer pre-activation vector
$W^{(l)}$	Layer weight matrix
$b^{(l)}$	Layer bias vector
$\sigma^{(l)}(\cdot)$	Non-linear activation function
$\tanh(\cdot)$	Hyperbolic tangent activation function
$[z]_+$	Rectified linear unit (ReLU) activation
$\hat{y}$	Network prediction vector
y	Target output vector
θ	Trainable network parameters
$\theta^\star$	Optimal network parameters
$\mathcal{D}$	Training dataset
$\mathcal{B}$	Mini-batch
$\mathcal{L}$	Total training loss function
$\mathcal{L}_{\text{MSE}}$	Mean squared error loss
$\mathcal{L}_{\text{reg}}$	Regularization loss
$\mathcal{L}_{\text{aux}}$	Auxiliary loss
λ_{reg}	Regularization weight
λ_{aux}	Auxiliary weight
$\delta^{(l)}$	Loss sensitivity per layer
$\delta^{(L)}$	Loss sensitivity at last layer
$\odot$	Element-wise (Hadamard) product
$\nabla_\theta \mathcal{L}$	Loss gradient with respect to parameters
η	Learning rate
E	Total epoch count

Symbol	Description
t	Optimization timestep index
μ_t	Biased first moment estimate
ν_t	Biased second moment estimate
$\hat{\mu}_t$	Bias-corrected first moment estimate
$\hat{\nu}_t$	Bias-corrected second moment estimate
β_1, β_2	Moment exponential decay rates
ϵ	Optimizer numerical stability constant
$x_{\text{adv}}^{(0)}$	Initial adversarial candidate state
$x_{\text{adv}}^{(j)}$	Adversarial candidate state at PGD step j
j	PGD iteration index
I	Maximum PGD iteration count
η_{PGD}	PGD step size
δ	Adversarial perturbation vector
ε_δ	Maximum perturbation magnitude
p	Norm order
$\mathcal{P}$	Admissible perturbation set
$\mathcal{J}(x, y; \theta)$	Adversarial loss function
$\Pi_{x+\mathcal{P}}(\cdot)$	Projection operator onto perturbation set
$\text{sign}(\cdot)$	Signum function
Neural Network Verification (Foundations)	
$\mathcal{F}$	Verification input domain
$\underline{x}_F, \overline{x}_F$	Lower and upper bounds of domain $\mathcal{F}$
$\psi(a^{(L)})$	Scalar specification function
$\psi^\star$	Global minimum of specification function
$\underline{\psi}$	Guaranteed sound specification lower bound
$[l_j^{(l)}, u_j^{(l)}]$	Pre-activation lower and upper bounds of neuron j in layer l
$\alpha_{L,j}^{(l)}, \alpha_{U,j}^{(l)}$	CROWN linear lower and upper bounding slopes
$d_{L,j}^{(l)}, d_{U,j}^{(l)}$	CROWN linear lower and upper bounding intercepts
$\zeta_l^+, \zeta_l^-, \zeta_l^\pm$	Strictly concave, convex, and mixed curvature neuron sets
$z^\star$	Adaptive tangent contact point
$\underline{\psi}_{\text{aff}}(x)$	Affine lower bounding function
$w_{\text{CROWN}}, c_{\text{CROWN}}$	Explicit CROWN linear bounding vector and constant
$\underline{\psi}_{\text{CROWN}}^\star$	Certified lower bound via CROWN
α	Lower-bound relaxation slopes
N_{unstable}	Total number of unstable neurons
$w(\alpha), c(\alpha)$	Parameterized affine bounding vector and constant
$\mathcal{G}(\alpha)$	Lower bound objective function in α-CROWN
$\underline{\psi}_\alpha^\star$	Optimized lower bound via α-CROWN
$\underline{\psi}_{\text{sub}}$	Specification lower bound on BaB subdomain
$\psi_{\text{sub}}^\star$	Minimum specification value on BaB subdomain
$\mathcal{F}_{\text{sub}}$	Input subdomain in Branch-and-Bound
β	Lagrange multipliers for neuron split constraints
$w(\alpha, \beta), c(\alpha, \beta)$	Parameterized surrogate bounding vector and constant

Symbol	Description
$\mathcal{G}(\alpha, \beta)$	Joint lower bound objective function in α,β-CROWN
$\underline{\psi}^{\star}_{\alpha,\beta}$	Jointly optimized lower bound via α,β-CROWN
Imitation Learning (Methodology)	
$\mathcal{X}_{\text{MPC}}$	MPC trajectory sampling domain
$\underline{x}_{\text{MPC}}$	Lower bound of MPC sampling domain
$\overline{x}_{\text{MPC}}$	Upper bound of MPC sampling domain
$\mathcal{X}_{\pi}$	Imitation learning state space
$\underline{x}_{\pi}$	Admissible state lower bound
$\overline{x}_{\pi}$	Admissible state upper bound
$\mathcal{D}_{\pi}$	Imitation learning dataset
$N_{\mathcal{D}}$	Imitation learning dataset sample count
$(x^{(i)}, u^{(i)})$	State-action sample pair in dataset
$\mathcal{B}_{\pi}$	Imitation learning training batch
$\pi_0(x)$	Expert MPC policy
$\pi_\theta(x)$	Learned imitation policy
$h_\theta(x)$	Unconstrained neural network output
clamp$(\cdot)$	Control input clamping function
S_x	State scaling matrix
S_u	Input scaling matrix
x_{scale}	State scaling vector
u_{scale}	Input scaling vector
$d(x)$	Scaled equilibrium distance
$w_x(x)$	Behavioral cloning loss weight
w_{min}	Baseline behavioral cloning loss weight
γ_x	Decay rate for state-dependent weight
$\mathcal{L}_{\text{BC}}$	Behavioral cloning loss
$\mathcal{L}_f$	Dynamics-aware constraint loss
λ_{BC}	Behavioral cloning loss weight
λ_f	Dynamics-aware constraint loss weight
$\mathcal{L}_{\pi}$	Total imitation policy loss
Neural Lyapunov Learning & Falsification	
$\pi_V(x)$	Co-trained imitation policy
$V_\phi(x)$	Neural Lyapunov candidate
$g_\phi(x)$	Neural network component of Lyapunov candidate
ϕ	Neural Lyapunov candidate parameters
ε	Regularization constant
R_ϕ	Learnable transformation matrix
ρ	Target Lyapunov sublevel value
ρ_{max}	Theoretical maximum sublevel bound
$\mathcal{V}_\rho$	Certified sublevel set
$\mathcal{X}_V$	Lyapunov training domain
$\underline{x}_V$	Lower bound of Lyapunov training domain
$\overline{x}_V$	Upper bound of Lyapunov training domain

Symbol	Description
$\partial\mathcal{X}_V$	Boundary of Lyapunov training domain
ρ_{min}	Minimum sublevel target bound
e_V	Lyapunov decrease violation
κ	Target decrease rate
ε_{rel}	Numerical stabilization constant
$\mathcal{L}_{\text{dec}}$	Relative Lyapunov decrease loss
S_{range}	Domain range scaling matrix
$\mathcal{L}_{\text{inv}}$	Relative state-invariance loss
λ_{inv}	Relative state-invariance loss weight
$\mathcal{L}_{\text{cond}}$	Combined condition violation loss
w_ρ	Soft sublevel gating weight
s	Gate sharpness parameter
$\mathcal{L}_{\rho-\text{gated}}$	Soft-gated condition loss
$\mathcal{B}_\rho$	Condition loss training batch
$\mathcal{B}_\partial$	Boundary candidate sample set
$\Pi_{\partial\mathcal{X}_V}$	Boundary projection operator
$\Pi_{\mathcal{X}_V}$	Domain projection operator
c_j	Active boundary limit along dimension j
$\mathcal{B}_{\text{ROA}}$	ROA shell sample set
ξ	Inner boundary shell factor
$\mathcal{L}_{\text{ROA}}$	Region of Attraction expansion loss
x_{center}	Center state of LIRPA hyper-rectangular region
$e_{V,\text{lower}}$	Signed Lyapunov decrease lower bound
$\mathcal{M}_{\text{L}}$	Region LIRPA condition violation
w_{L}	LIRPA region weight
γ_{L}	LIRPA weight decay rate
N_{L}	LIRPA region count
$\mathcal{L}_{\text{L}}$	LIRPA loss
$\mathcal{B}_V$	Lyapunov scaling Sobol sample batch
ℓ_V	Mean Lyapunov log-value
$\ell_{V,0}$	Initial mean Lyapunov log-value
$\mathcal{L}_V$	Lyapunov magnitude scaling loss
$\mathcal{B}_{\text{pol}}$	Policy scaling Sobol sample batch
$\mathcal{L}_{\text{pol}}$	Policy scaling loss
$\ell_{R_\phi,0}$	Initial matrix R_ϕ Frobenius norm
$\mathcal{L}_{R_\phi}$	Matrix R_ϕ regularization loss
$\mathcal{L}_{\text{total}}$	Total combined Lyapunov loss
$\lambda_m, \mathcal{L}_m$	Generic indexed loss weight and component
$\lambda_{\rho-\text{gated}}$	Soft-gated condition loss weight
λ_{ROA}	Region of Attraction expansion loss weight
λ_{L}	LIRPA loss weight
λ_V	Lyapunov magnitude scaling loss weight
λ_{pol}	Policy scaling loss weight
λ_{R_ϕ}	Matrix R_ϕ regularization loss weight
$\mathcal{C}$	Counterexample pool

Symbol	Description
$\mathcal{S}$	Explorative pool
$\mathcal{B}_{\mathcal{S}}$	Explorative pool candidate sample set
$N_{\mathcal{S}}$	Explorative pool target size
x_{adv}	Counterexample state
τ	Counterexample age
$\tau_{\max}$	Maximum lifespan
N_{mined}	Number of newly mined counterexamples
r_{mine}	Empirical counterexample mining yield
$\tilde{r}_{\text{mine}}$	Smoothed counterexample yield EMA
$\upsilon_{\mathcal{C}}$	Counterexample yield EMA decay rate
$r_{\mathcal{C}}$	Effective counterexample batch fraction
$r_{\min}, r_{\max}$	Minimal and maximal counterexample batch fraction bounds
m_ρ	Sublevel margin
χ	Overestimation factor
$\tilde{\rho}$	Estimated ROA boundary minimum
υ_ρ	ρ EMA decay rate
N_{outer}	Outer epoch count
K_{steps}	Inner mini-batch optimization steps per epoch
Formal Stability Verification (Methodology)	
$\mathcal{X}_C$	Formal verification domain
$\underline{x}_C, \overline{x}_C$	Lower and upper bounds of verification domain
$\mathcal{X}_{\text{hole}}$	Equilibrium exclusion region
$\mathcal{X}_{\text{eval}}$	Punctured evaluation domain
$\Phi(x;\rho)$	Verification specification formula
Φ_{sub}	Sublevel set exclusion predicate
Φ_{pos}	Positive definiteness predicate
Φ_{inv}	Forward invariance predicate
Φ_{dec}	Strict Lyapunov decrease predicate
$\Psi(\rho)$	Global domain verification condition
ρ_0	Initial sublevel search bound
$\rho_{\text{lo}}, \rho_{\text{up}}$	Search interval lower and upper bounds
q	Exponential search scaling factor
$N_{\text{scale}}, N_{\text{bisect}}$	Scaling and bisection iteration limits
$\rho^\star$	Maximal verified sublevel value
Benchmark Systems & Experimental Evaluation	
Δt	Discrete simulation sampling time
f_{DI}	Discrete-time double integrator dynamics
f_{CP}	Discrete-time cartpole dynamics
p, v	Cart position and velocity states
φ, ω	Cartpole pendulum angle and angular velocity states
F	Control force applied to the cart
M	Cart mass
m	Pendulum mass

Symbol	Description
l_p	Pendulum length
g	Acceleration due to gravity

1 Introduction

The use of autonomous cyber-physical systems in safety-critical areas requires control architectures that combine high flexibility, real-time capability, and strict safety guarantees. Model Predictive Control (MPC) is well-established for constrained multivariable optimal control, offering systematic and provable constraint satisfaction under nominal model assumptions. Furthermore, MPC provides well-understood stability guarantees for the closed-loop system, based on Lyapunov theory [1], [2]. Even though numerical solvers become increasingly efficient, real-time execution of nonlinear MPC requires a significant amount of computational power, which is especially limiting for embedded systems. To address this computational limitation, neural control policies synthesized using Imitation or Reinforcement Learning offer highly efficient inference during a forward pass [3]–[5], motivating recent interest in synergistic MPC-learning frameworks [6].

Despite their remarkable performance on high-dimensional and nonlinear tasks, the implementation of trained neural control strategies in safety-critical areas remains severely limited. In industrial and public applications, deployment rarely fail due to lack of performance, but rather due to the absence of rigorous, verifiable guarantees regarding closed-loop stability, forward invariance, and robustness [7]. Standard neural networks generalize well on average, but are vulnerable to out-of-distribution shifts, domain violations, and adversarial perturbations [8]–[11].

Consequently, formal guarantees are no longer just a theoretical problem, as lawmakers and regulatory agencies around the world are shifting towards binding technical requirements. Under the European Union's Artificial Intelligence Act (Regulation (EU) 2024/1689), autonomous and safety-critical control systems fall into the category of *High-Risk AI*, which imposes strict regulatory requirements on risk management, system robustness and verifiable reliability [12]. Since the regulatory frameworks requires compliance without prescribing specific algorithmic solutions, this work addresses this gap by proposing an integrated, GPU-accelerated framework for training and formally verifying neural imitation policies using learned Lyapunov candidates.

State of the Art

In control theory, the stability of a closed-loop system is typically proven using Lyapunov stability theory, which requires finding a scalar, energy-like function $V(x)$ that is positive definite and strictly decreasing along non-equilibrium closed-loop trajectories over a Lyapunov sublevel set $\mathcal{V}_\rho$, which provides a certified inner approximation of the ROA [1], [2]. However, for nonlinear systems controlled by neural policies, it is difficult to construct these Lyapunov certificates and formally verify the Lyapunov conditions across a continuous state space. Recent research has therefore focused on the joint synthesis of neural Lyapunov functions and neural controllers, coupled with formal verification methods [7]. To verify these stability properties, established verification frameworks rely on general-purpose Satisfiability Modulo Theories

(SMT) [13] or Mixed-Integer Linear Programming (MILP) [14] solvers. While mathematically sound, they scale poorly with increasing state-space dimensionality and network depth, which makes the formal verification of nonlinear closed-loop systems on larger domains challenging [15], [16]. To address these computational constraints, recent advances in formal verification methods introduce complete, GPU-accelerated bound propagation using α,β-CROWN [16], [17]. By combining linear bound propagation with Branch-and-Bound verification, α,β-CROWN can substantially accelerate complete verification relative to classical LP-based branch-and-bound baselines, enabling formal certification for broader classes of closed-loop neural control benchmarks [7].

Contributions of this Work

In contrast to prior work focusing on learning certified neural controllers from scratch, this thesis establishes an integrated framework for learning imitation policies from nominal MPC experts, followed by learning a corresponding Lyapunov candidate to formally verify the converged policy. A dataset generated with an MPC expert is used to optimize a constrained policy architecture that enforces control input bounds $\underline{u} \leq \pi_\theta(x) \leq \overline{u}$ and satisfies the equilibrium constraint $\pi_\theta(x^\star) = u^\star$ by construction. The optimization penalizes deviations with state-dependent weights prioritizing accuracy near the origin, while promoting forward invariance using a one-step dynamics-aware loss. Building upon the converged imitation policy, the policy is refined while simultaneously learning a structurally positive definite neural Lyapunov candidate $V_\phi(x)$. The optimization objective incorporates a soft ρ-gated Lyapunov condition and state invariance loss, augmented by LiRPA bound propagation penalties to shape a verifier-friendly loss landscape. To compute the gated loss, the sublevel bound ρ is dynamically estimated along the domain boundary using ℓ_∞-Projected Gradient Descent (PGD). Simultaneously, adversarial mining using PGD actively searches for stability counterexamples, which are maintained in a replay buffer and fed back into training to actively address localized condition violations. For rigorous guarantees, a formal certification pipeline based on the complete α,β-CROWN verifier utilizes a scaling and bisection search algorithm to soundly compute the maximal certified sublevel value ρ^* [7]. Formulating the stability criteria as a disjunctive logical specification over a punctured state space enables the solver to formally certify positive definiteness, regional forward invariance, and exponential Lyapunov decrease.

Overview

The remainder of this thesis consists of seven chapters, with Chapter 2 presenting the theoretical foundations for Lyapunov stability as well as for model predictive control (MPC) under constraints and closed-loop stability. This is followed by Chapter 3, which explains the fundamentals of neural networks, in particular multilayer perceptrons, efficient optimization, and backpropagation, as well as adversarial attacks using Projected Gradient Descent (PGD). Chapter 4 covers formal neural network verification using α,β-CROWN, contrasting incomplete linear bound propagation with complete Branch-and-Bound methods. Chapter 5 details the proposed framework, covering the constrained imitation architecture, the Lyapunov learning pipeline, and the verified sublevel set search. Chapter 6 evaluates the framework on a linear double integrator and a nonlinear cartpole benchmark, including their setup. In Chapter 7, the resulting trade-offs and scalability limits are analyzed. Finally, Chapter 8 summarizes the essence of this work and outlines further research recommendations for certified neural control.

2 Optimal Control and Lyapunov Stability Theory

This chapter provides the theoretical foundation for discrete-time dynamical systems and Model Predictive Control (MPC), which serves as the expert policy for the imitation learning framework used in this work. First, Section 2.1 introduces discrete-time dynamical systems, state and input constraints, and the core principles of Lyapunov stability. Afterwards, Section 2.2 outlines the constrained finite-horizon optimal control problem and details the terminal ingredients required to guarantee asymptotic stability of the closed-loop system.

2.1 Discrete-Time Dynamical Systems and Stability

Implementing optimal control schemes on digital hardware requires a discrete-time representation. Consequently, the continuous-time dynamics

$$\dot{x}(t) = f_c(x(t), u(t)) \tag{2.1}$$

must be discretized, where $x(t) \in \mathbb{R}^{n_x}$ and $u(t) \in \mathbb{R}^{n_u}$ denote the continuous-time state and input vectors, respectively, with state space dimension $n_x \in \mathbb{N}$ and input dimension $n_u \in \mathbb{N}$. In this work, the corresponding discrete-time system

$$x_{k+1} = f(x_k, u_k) \tag{2.2}$$

is obtained by approximating the continuous dynamics using a numerical integration scheme with a constant sampling time $T_s > 0$. Here, $k \in \mathbb{N}_0$ denotes the discrete time step index, such that $x_k := x(kT_s) \in \mathbb{R}^{n_x}$ and $u_k := u(kT_s) \in \mathbb{R}^{n_u}$ represent the state and control input at time step k. The discrete-time function $f : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \to \mathbb{R}^{n_x}$ is evaluated using a numerical integration scheme, such as the forward Euler or a fourth-order Runge-Kutta (RK4) method [18].

To account for physical and operational limitations, the system states and control inputs are restricted to constraint sets $\mathcal{X} := [\underline{x}, \overline{x}] \subseteq \mathbb{R}^{n_x}$ and $\mathcal{U} := [\underline{u}, \overline{u}] \subseteq \mathbb{R}^{n_u}$. Furthermore, without loss of generality, any equilibrium $(x^\star, u^\star)$ can be shifted to the origin via deviation variables $\tilde{x} := x - x^\star$ and $\tilde{u} := u - u^\star$. For readability, the tilde notation is subsequently dropped, so that x and u represent the shifted variables, assuming $(x^\star, u^\star) = (\mathbf{0}, \mathbf{0})$ throughout this work.

2.1.1 Lyapunov Stability

Consider the discrete-time closed-loop system under a state-feedback control policy $\pi : \mathcal{X} \to \mathcal{U}$. The origin serves as an equilibrium point $x^\star = \mathbf{0}$ satisfying $\pi(\mathbf{0}) = \mathbf{0}$ and $f(\mathbf{0}, \mathbf{0}) = \mathbf{0}$. Assuming the compact

state constraint set $\mathcal{X}$ is positively forward invariant under π, the origin is locally asymptotically stable in $\mathcal{X}$ if there exists a continuous, positive definite function $V : \mathcal{X} \to \mathbb{R}_{\geq 0}$ satisfying[2, Theorem 2.13]:

$$\begin{aligned} V(\mathbf{0}) &= 0, \\ V(x) &> 0 \qquad \forall x \in \mathcal{X} \setminus \{\mathbf{0}\}, \\ V(f(x, \pi(x))) - V(x) &< 0 \qquad \forall x \in \mathcal{X} \setminus \{\mathbf{0}\}. \end{aligned} \tag{2.3}$$

Here, the decrease condition ensures that any trajectory starting in $\mathcal{X}$ remains bounded within $\mathcal{X}$ and asymptotically converges towards the equilibrium $x^\star = 0$.

2.2 Model Predictive Control and Stability

This section summarizes the model predictive control (MPC) framework as well as the conditions that are required to ensure the stability of the control loop in optimal control configurations with a finite time horizon.

2.2.1 Problem Formulation

For the nominal MPC setup considered in this work, the following finite-horizon optimal control problem is solved at each time step k given the current system state x_k[2, Sec. 2.2], [1, Sec. 3.3]:

$$\begin{aligned} \min_{(x_{i|k})_{i=0}^{N}, (u_{i|k})_{i=0}^{N-1}} \quad & J\left(x_k, (u_{i|k})_{i=0}^{N-1}\right) = \sum_{i=0}^{N-1} \ell(x_{i|k}, u_{i|k}) + V_f(x_{N|k}) \\ \text{s.t.} \quad & x_{i+1|k} = f(x_{i|k}, u_{i|k}), && i = 0, \dots, N-1, \\ & x_{i|k} \in \mathcal{X}, && i = 0, \dots, N-1, \\ & u_{i|k} \in \mathcal{U}, && i = 0, \dots, N-1, \\ & x_{N|k} \in \mathcal{X}_f, \\ & x_{0|k} = x_k. \end{aligned} \tag{2.4}$$

Here, $N \in \mathbb{N}$ denotes the prediction horizon, $\ell : \mathcal{X} \times \mathcal{U} \to \mathbb{R}_{\geq 0}$ is the stage cost function, and $V_f : \mathcal{X}_f \to \mathbb{R}_{\geq 0}$ represents the terminal cost function, which is defined over the terminal constraint set $\mathcal{X}_f \subseteq \mathcal{X}$. To distinguish optimization from closed-loop variables, $x_{i|k}$ and $u_{i|k}$ denote the predicted state and input at horizon step i, initialized at sampling time k. The sequences $(x_{i|k})_{i=0}^{N}$ and $(u_{i|k})_{i=0}^{N-1}$ serve as optimization variables constrained by the sets $\mathcal{X}$ and $\mathcal{U}$[1, Sec. 3.2]. Furthermore, the optimal value function is defined as $V_N(x_k) := J\left(x_k, (u^\star_{i|k})_{i=0}^{N-1}\right)$.

For this work, the stage cost is chosen as the quadratic cost function with positive definite weighting matrices $Q \succ 0$ and $R \succ 0$, which is defined as follows[2, Sec. 1.3]:

$$\ell(x_{i|k}, u_{i|k}) = x_{i|k}^\top Q x_{i|k} + u_{i|k}^\top R u_{i|k}. \tag{2.5}$$

Following the receding horizon principle, only the first optimal control input $u^\star_{0|k}$ of the sequence is applied to the system. This implicitly defines the closed-loop MPC feedback policy $\pi_0 : \mathcal{X}_N \to \mathcal{U}$ as

$$\pi_0(x_k) := u^\star_{0|k}, \tag{2.6}$$

yielding the nominal closed-loop dynamics $x_{k+1} = f(x_k, \pi_0(x_k))$. The optimal control problem (2.4) is then repeated at the subsequent step with the updated state[2, Sec. 2.2], [1, Sec. 3.1].

2.2.2 Terminal Ingredients and their Construction

To establish recursive feasibility and closed-loop stability of nominal MPC, the optimal control problem (2.4) incorporates three central design ingredients: the terminal constraint set $\mathcal{X}_f \subseteq \mathcal{X}$, an auxiliary local control law $\pi_f : \mathcal{X}_f \to \mathcal{U}$, and the terminal cost function $V_f : \mathcal{X}_f \to \mathbb{R}_{\geq 0}$ [2, Assumption 2.14], [1, Assumption 5.9].

Specifically, these terminal ingredients must satisfy state and input constraints, positive invariance of the terminal set, and local Lyapunov decrease of the terminal cost:

$$\mathcal{X}_f \subseteq \mathcal{X}, \tag{2.7a}$$
$$\pi_f(x) \in \mathcal{U}, \qquad \forall x \in \mathcal{X}_f, \tag{2.7b}$$
$$f(x, \pi_f(x)) \in \mathcal{X}_f, \qquad \forall x \in \mathcal{X}_f, \tag{2.7c}$$
$$V_f(f(x, \pi_f(x))) - V_f(x) \leq -\ell(x, \pi_f(x)), \qquad \forall x \in \mathcal{X}_f. \tag{2.7d}$$

The conditions (2.7a) and (2.7b) ensure that terminal states and their corresponding local control inputs remain within the admissible state and input bounds $\mathcal{X}$ and $\mathcal{U}$. Additionally, positive invariance in (2.7c) ensures that every state in $\mathcal{X}_f$ remains in $\mathcal{X}_f$ under the local controller $\pi_f(x)$[2, Def. 2.9]. This property is essential for proving recursive feasibility, as it enables the terminal state of a feasible solution at horizon N to be extended by the local control input $\pi_f(x^\star_{N|k})$. This yields a feasible candidate trajectory for the subsequent sampling step $k+1$, as further described in Section 2.2.3[2, Sec. 2.4.2]. The terminal cost decrease condition in (2.7d) acts as a local Lyapunov condition within $\mathcal{X}_f$. While positive invariance ensures that the local trajectory remains in $\mathcal{X}_f$, the decrease condition requires the terminal cost V_f to decrease along the local closed-loop trajectory by at least the stage cost $\ell(x,\pi_f(x))$ [1, Assumption 5.9].

The predictive mechanism and the geometric role of the terminal set are illustrated in Fig. 2.1. While intermediate predicted states are required to remain in the safe state space $x_{i|k} \in \mathcal{X}$ for $i = 0, \ldots, N-1$, the terminal constraint enforces $x_{N|k} \in \mathcal{X}_f$, where the local controller satisfies the Lyapunov decrease condition in (2.7d) [2, Theorem 2.19].

For nonlinear systems, the terminal ingredients are typically designed by using a linearized discrete-time model, which can be obtained by linearizing the system dynamics around the equilibrium point $(x, u) = (\mathbf{0}, \mathbf{0})$, yielding

$$x_{k+1} = Ax_k + Bu_k, \tag{2.8}$$

where $A = \frac{\partial f}{\partial x}(\mathbf{0}, \mathbf{0})$ and $B = \frac{\partial f}{\partial u}(\mathbf{0}, \mathbf{0})$ represent the system Jacobians[2, Sec. 3.5]. Assuming (A, B) is stabilizable and $(A, Q^{1/2})$ is detectable, the matrix $P \succ 0$ is obtained as the unique stabilizing solution to the discrete-time Algebraic Riccati Equation (DARE)[2, Eq. (1.18)], [1, Eq. (5.23)]:

$$P = A^\top PA - A^\top PB \left(R + B^\top PB\right)^{-1} B^\top PA + Q. \tag{2.9}$$

Using P, the local linear feedback law $\pi_f(x) = -Kx$ is parameterized via the optimal Linear-Quadratic Regulator (LQR) gain [2, Eq. (1.18)], [1, Eq. (5.24)]:

$$K = \left(R + B^\top PB\right)^{-1} B^\top PA, \tag{2.10}$$

and the terminal cost function is specified quadratically as $V_f(x) = x^\top P x$. Accordingly, the terminal set $\mathcal{X}_f$ is defined in this specific case as an ellipsoidal sub-level set of this cost function [2, Sec. 3.5]:

$$\mathcal{X}_f := \left\{ x \in \mathbb{R}^{n_x} \;\middle|\; x^\top P x \leq \rho \right\}. \tag{2.11}$$

By choosing the scaling parameter $\rho > 0$ sufficiently small, $\mathcal{X}_f$ satisfies both state and input constraints $\mathcal{X}$ and $\mathcal{U}$, while ensuring that the quadratic Lyapunov decrease dominates higher-order linearization errors of the underlying nonlinear dynamics.

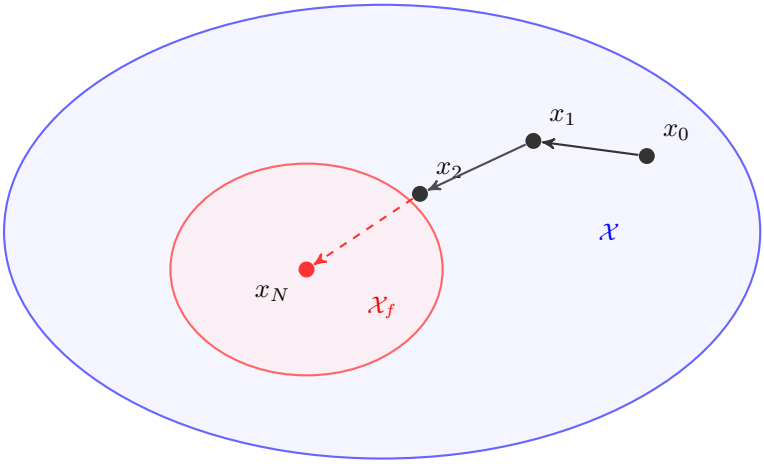

Figure 2.1: Comparison of the terminal set $\mathcal{X}_f$ (red) with the entire state space $\mathcal{X}$ (blue).

2.2.3 Recursive Feasibility and Closed-Loop Stability

Due to state and input constraints, as well as system dynamics, the optimal control problem (OCP) may be infeasible for arbitrary states $x \in \mathcal{X}$. Therefore, the MPC domain is restricted to the *feasible set*, which is defined as[1, Sec. 3.3]:

$$\mathcal{X}_N := \left\{ x_0 \in \mathcal{X} \;\middle|\; \exists (u_i)_{i=0}^{N-1} \in \mathcal{U}^N \;:\; x_{i+1} = f(x_i, u_i) \in \mathcal{X} \; \forall i = 0, \ldots, N-1,\; x_N \in \mathcal{X}_f \right\}. \tag{2.12}$$

For any state $x_k \in \mathcal{X}_N$, solving (2.4) yields the optimal input sequence $(u^\star_{i|k})_{i=0}^{N-1}$ and the corresponding optimal state sequence $(x^\star_{i|k})_{i=0}^{N}$, where $x^\star_{0|k} = x_k$ and $x^\star_{N|k} \in \mathcal{X}_f$. Under the MPC policy π_0 from (2.6), the nominal closed-loop system yields the successor state $x_{k+1} = f(x_k, \pi_0(x_k)) = x^\star_{1|k}$.

Recursive Feasibility with Sequence Shifting. In the nominal closed-loop setup, an MPC formulation is said to be *recursively feasible* if (2.4) is feasible for $x_k \in \mathcal{X}_N$, resulting in a subsequent state x_{k+1} that is also feasible. This can be demonstrated with the shifted candidate input sequence $(\tilde{u}_{i|k+1})_{i=0}^{N-1}$ constructed for the state $x_{k+1} = x^\star_{1|k}$, where the terminal controller π_f is appended:

$$(\tilde{u}_{i|k+1})_{i=0}^{N-1} := \left(u^\star_{1|k}, \ldots, u^\star_{N-1|k}, \pi_f(x^\star_{N|k}) \right). \tag{2.13}$$

The corresponding predicted state sequence is given by

$$(\tilde{x}_{i|k+1})_{i=0}^{N} := \left(x^\star_{1|k}, \ldots, x^\star_{N|k}, f(x^\star_{N|k}, \pi_f(x^\star_{N|k})) \right). \tag{2.14}$$

For $i = 0, \dots, N-2$, the pairs $(\tilde{x}_{i|k+1}, \tilde{u}_{i|k+1}) = (x^\star_{i+1|k}, u^\star_{i+1|k})$ are feasible by construction. Additionally, for $i = N-1$, the input $\tilde{u}_{N-1|k+1} = \pi_f(x^\star_{N|k})$ is admissible according to (2.7b), since $x^\star_{N|k} \in \mathcal{X}_f$. The computed terminal state satisfies $\tilde{x}_{N|k+1} = f(x^\star_{N|k}, \pi_f(x^\star_{N|k})) \in \mathcal{X}_f \subseteq \mathcal{X}$ due to the positive invariance of $\mathcal{X}_f$ in (2.7c). Thus, the control sequence $(\tilde{u}_{i|k+1})_{i=0}^{N-1}$ is feasible for x_{k+1}, which proves that $x_{k+1} \in \mathcal{X}_N$. Consequently, for all future time steps $k \geq 0$, the closed-loop state remains in the feasible set $\mathcal{X}_N$, which confirms both positive invariance of $\mathcal{X}_N$ under π_0 and recursive feasibility.

Lyapunov Decrease of the Optimal Value Function. As the shifted candidate input sequence is feasible for x_{k+1}, the optimal cost satisfies $V_N(x_{k+1}) \leq J(x_{k+1}, (\tilde{u}_{i|k+1})_{i=0}^{N-1})$. Subtracting the optimal cost at step k from the candidate cost at time step $k+1$ yields:

$$\begin{aligned} &J\left(x_{k+1}, (\tilde{u}_{i|k+1})_{i=0}^{N-1}\right) - V_N(x_k) \\ &\quad = \sum_{i=1}^{N-1} \ell(x^\star_{i|k}, u^\star_{i|k}) + \ell(x^\star_{N|k}, \pi_f(x^\star_{N|k})) + V_f\left(f(x^\star_{N|k}, \pi_f(x^\star_{N|k}))\right) \\ &\qquad - \left[\ell(x_k, u^\star_{0|k}) + \sum_{i=1}^{N-1} \ell(x^\star_{i|k}, u^\star_{i|k}) + V_f(x^\star_{N|k})\right] \\ &\quad = -\ell(x_k, u^\star_{0|k}) + \underbrace{\left[V_f(f(x^\star_{N|k}, \pi_f(x^\star_{N|k}))) - V_f(x^\star_{N|k}) + \ell(x^\star_{N|k}, \pi_f(x^\star_{N|k}))\right]}_{\leq 0 \text{ due to (2.7d)}}. \end{aligned} \tag{2.15}$$

Applying the local Lyapunov decrease condition (2.7d) ensures that the bracketed term is non-positive [1, Assumption 5.9]. Together with $V_N(x_{k+1}) \leq J(x_{k+1}, (\tilde{u}_{i|k+1})_{i=0}^{N-1})$, the optimal value function satisfies the Lyapunov decrease condition along the nominal closed-loop trajectories[1, Lemma 5.12]:

$$V_N(x_{k+1}) - V_N(x_k) \leq -\ell(x_k, \pi_0(x_k)), \qquad \forall x_k \in \mathcal{X}_N. \tag{2.16}$$

Asymptotic Stability and Region of Attraction. To prove asymptotic stability of the origin, the optimal value function V_N must satisfy the criteria of a Lyapunov function on the feasible set $\mathcal{X}_N$[1, Sec. 2.3]. Given that $Q \succ 0$, the minimum eigenvalue satisfies $\lambda_{\min}(Q) > 0$. Together with $u^\top R u \geq 0$, this results in a strict lower bound on the stage cost (2.5):

$$\ell(x, u) \geq x^\top Q x \geq \lambda_{\min}(Q)\|x\|^2 > 0, \qquad \forall x \neq \mathbf{0}. \tag{2.17}$$

Consequently, this yields a negative upper bound on the Lyapunov decrease in (2.16):

$$V_N(x_{k+1}) - V_N(x_k) \leq -\ell(x_k, \pi_0(x_k)) \leq -\lambda_{\min}(Q)\|x_k\|^2 < 0, \qquad \forall x_k \in \mathcal{X}_N \setminus \{\mathbf{0}\}. \tag{2.18}$$

Furthermore, the optimal value function satisfies $V_N(0) = 0$ and the lower bound $V_N(x) \geq \ell(x, \pi_0(x)) \geq \lambda_{\min}(Q)\|x\|^2 > 0$ for all $x \in \mathcal{X}_N \setminus \{0\}$. Under the assumption that V_N is continuous at the origin, V_N is strictly positive definite on $\mathcal{X}_N$[1, Def. 2.13].

In combination with the positive forward invariance of $\mathcal{X}_N$ established by recursive feasibility, V_N defines a valid Lyapunov function. Therefore, the origin $\mathbf{0}$ of the nominal closed-loop system is asymptotically stable, and every trajectory initialized at $x_0 \in \mathcal{X}_N$ converges to the origin, making the feasible set $\mathcal{X}_N$ the *region of attraction*[2, Theorem 2.19], [1, Theorem 5.13].

3 Foundations of Neural Control Policies

In imitation learning, artificial neural networks (ANNs) approximate complex control laws offline to bypass the computational overhead of online Model Predictive Control (MPC) evaluations. As the machine learning foundation for this work, Section 3.1 introduces the basic architecture of Multi-Layer Perceptrons (MLPs), loss functions, backpropagation, and gradient-based optimization methods. Afterwards, Section 3.2 presents adversarial example generation and Projected Gradient Descent (PGD), which serve to identify counterexamples to the Lyapunov conditions.

3.1 Artificial Neural Networks

3.1.1 Multi-Layer Perceptrons

Multi-layer perceptrons (MLPs) are fully connected feed-forward networks that map an input through $L-1$ hidden layers to an output layer [8, Sec. 1.4], [19, Sec. 3.1]. Let $n_l \in \mathbb{N}$ denote the number of neurons in layer $l \in \{0, \dots, L\}$, where n_0 represents the input dimension, n_L the output dimension, and n_h denotes the number of neurons per hidden layer, with $n_l = n_h$ for all $l \in \{1, \dots, L-1\}$. For an input vector $x \in \mathbb{R}^{n_0}$, forward propagation is defined recursively for layers $l = 1, \dots, L$:

$$a^{(l)} = \sigma^{(l)}\left(z^{(l)}\right) \quad \text{with} \quad z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad \forall l = 1, \dots, L, \tag{3.1}$$

where $a^{(0)} := x$ is the network input, $z^{(l)} \in \mathbb{R}^{n_l}$ is the pre-activation vector of layer l, and $a^{(L)} := \hat{y} \in \mathbb{R}^{n_L}$ is the final network prediction. The matrix $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ and vector $b^{(l)} \in \mathbb{R}^{n_l}$ denote the weight matrix and bias vector of layer l, respectively, while the function $\sigma^{(l)}(\cdot)$ represents an element-wise activation function [19, Sec. 3.2]. For regression and control tasks, the output layer is typically chosen as the identity mapping $\sigma^{(L)}(z^{(L)}) = z^{(L)}$, whereas hidden layers utilize non-linear activations $\sigma^{(l)}$ for $l \in \{1, \dots, L-1\}$. Without these non-linearities, the layers would simply be reduced to a single affine transformation, drastically limiting the model's ability to capture complexity[8, Sec. 1.5]. Common activation functions in neural control include the hyperbolic tangent (3.2a) and the rectified linear unit (ReLU) (3.2b), defined respectively as[19, Sec. 3.5]:

$$\tanh z = \frac{\mathrm{e}^{2z} - 1}{\mathrm{e}^{2z} + 1} \tag{3.2a}$$

$$[z]_+ = \max(0, z). \tag{3.2b}$$

Their qualitative profiles are illustrated in Fig. 3.1.

Based on the universal approximation theorem, an MLP with at least one sufficiently sized hidden layer and non-linear activations can approximate any continuous multidimensional function on compact sets

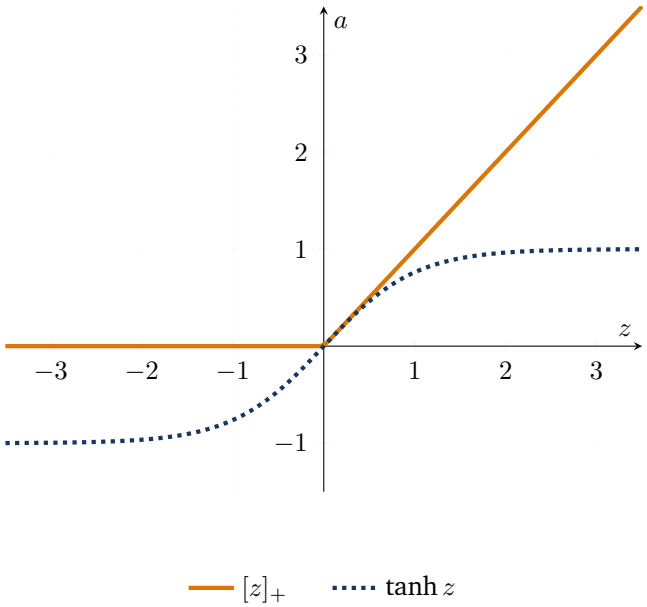

Figure 3.1: Standard activation functions for neural control policies.

to arbitrary accuracy [19, Sec. 3.1]. The trainable parameters are represented by $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^{L}$. Figure 3.2 illustrates a representative architecture.

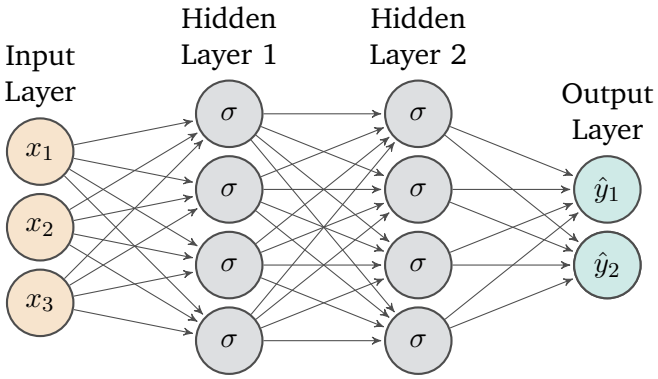

Figure 3.2: Architecture of an exemplary Multi-Layer Perceptron with three inputs, two outputs and two hidden layers with four neurons each. Illustration adapted from[19, Fig. 3.1]

3.1.2 Optimization and Training

To approximate an expert control law with a neural network $\pi_\theta(x)$, the parameters θ are updated via gradient-based optimization[19, Sec. 2.5]. The objective is to find parameters $\theta^\star$ that minimize the empirical training objective:

$$\theta^\star \in \arg\min_\theta \mathcal{L}(\mathcal{D}; \theta). \tag{3.3}$$

Dataset and Primary Loss Function. The training process uses a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{|\mathcal{D}|}$ containing $|\mathcal{D}|$ input-target pairs sampled across the domain. For a continuous regression tasks, a common loss term is the mean squared error (MSE) evaluated over the training dataset:

$$\mathcal{L}_{\text{MSE}}(\mathcal{D}; \theta) = \frac{1}{|\mathcal{D}|} \sum_{(x,y)\in\mathcal{D}} \|\pi_\theta(x) - y\|_2^2 . \tag{3.4}$$

Here, x and y denote an input sample and its corresponding target, respectively. Minimizing this objective penalizes prediction deviations, thereby encouraging the network to accurately capture the underlying mapping.

Loss Augmentation and Regularization. In supervised learning, focusing exclusively on minimizing empirical error over a finite dataset can lead to overfitting. Furthermore, it may fail to enforce underlying structural or physical properties. To improve generalization and incorporate prior knowledge directly into the optimization process, the objective is frequently augmented with weight regularization and auxiliary loss terms[19, Sec. 3.7]:

$$\mathcal{L}(\mathcal{D}; \theta) = \mathcal{L}_{\text{MSE}}(\mathcal{D}; \theta) + \lambda_{\text{reg}}\mathcal{L}_{\text{reg}}(\theta) + \lambda_{\text{aux}}\mathcal{L}_{\text{aux}}(\mathcal{D}; \theta). \tag{3.5}$$

where $\lambda_{\text{reg}}, \lambda_{\text{aux}} \geq 0$ are utilized as weights balancing prediction error, parameter regularization, and constraint satisfaction. The regularization term $\mathcal{L}_{\text{reg}}$ constrains the network's parameter space, typically realized as an ℓ_1-norm ($\|\theta\|_1$) or ℓ_2-norm ($\|\theta\|_2^2$) penalty. While ℓ_1-regularization promotes sparse parameter representations, ℓ_2-regularization penalizes large parameter magnitudes and can improve numerical stability and generalization. In practice, these penalties are applied exclusively to the network's weight matrices $W \subset \theta$, leaving bias vectors b unpenalized.[19, Sec. 3.7.2]. Alongside regularization penalties, the auxiliary loss $\mathcal{L}_{\text{aux}}$ incorporates domain knowledge, such as differential equations or Lyapunov conditions[19, Sec. 5], [7], into the optimization process. Enforcing these properties during training helps the network learn the desired physically sound solutions.

Gradient Computation via Backpropagation. To update the network parameters θ using gradient-based optimization, the partial derivatives of the loss $\mathcal{L}(\mathcal{D}, \theta)$ with respect to all weight matrices $W^{(l)}$ and bias vectors $b^{(l)}$ are calculated. Backpropagation efficiently computes chain-rule derivatives of loss components that depend on the parameters through the network output by combining a forward pass (3.1) with a backward pass[19, Sec. 3.4].

During the backward pass, the algorithm computes the sensitivity of the loss with respect to the pre-activations, defined as $\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l)}}$. At the output layer L, this sensitivity is initialized as:

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \odot \frac{\partial a^{(L)}}{\partial z^{(L)}}, \tag{3.6}$$

where $\odot$ represents the element-wise (Hadamard) product. Propagating the sensitivity backward for $l = L-1, \ldots, 1$ results in the recursive equation:

$$\delta^{(l)} = \left((W^{(l+1)})^T \delta^{(l+1)}\right) \odot \frac{\partial a^{(l)}}{\partial z^{(l)}} \tag{3.7}$$

Combining these loss sensitivitys with stored activations $a^{(l-1)}$ yields the network-dependent gradients of the output-dependent loss terms. Adding the explicit derivative of the parameter regularization term $\mathcal{L}_{\text{reg}}$ completes the total gradient evaluation:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)}\right)^T + \lambda_{\text{reg}} \frac{\partial \mathcal{L}_{\text{reg}}}{\partial W^{(l)}}, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}. \tag{3.8}$$

By reusing intermediate activations and sensitivities, backpropagation avoids redundant derivative calculations and computes parameter gradients with a computational cost similar to that of a forward pass through the network[19, Sec. 3.4], [8, Sec. 2.3].

Optimization via Mini-Batch Gradient Descent. While full-batch gradient descent evaluates the loss gradient over the entire dataset $\mathcal{D}$, this can become computationally expensive and impractical for large dataset sizes $|\mathcal{D}|$. Conversely, single-sample stochastic gradient descent (SGD) results in high variance of the parameter updates, leading to erratic optimization trajectories. Mini-batch gradient descent solves this problem by dividing the dataset into smaller subsets $\mathcal{B} \subset \mathcal{D}$ of size $|\mathcal{B}|$. For a mini-batch, the loss $\mathcal{L}(\mathcal{B}; \theta)$ is evaluated analogously to Eq. (3.5), with $\mathcal{D}$ replaced by $\mathcal{B}$ in all data-dependent loss terms. Typically, the mini-batch gradient $\nabla_\theta \mathcal{L}(\mathcal{B}; \theta)$ has lower variance than the gradient obtained from a single sample, while mini-batch processing enables efficient parallelization on modern GPU hardware. During training, the parameters θ are iteratively updated following gradient descent, scaled by a learning rate $\eta > 0$. This update loop is in turn executed for a total of $E \in \mathbb{N}$ epochs [8, Sec. 2.6.1], [19, Sec. 2.6].

Adaptive Moment Estimation. Although mini-batch gradient descent improves stability compared to SGD, its performance depends on a static learning rate η. In loss landscapes with noisy mini-batch gradients or challenging curvature, a fixed learning rate can cause parameter updates to oscillate in steep directions while making slow progress or none on plateaus[8, Sec. 4.5.4].

To resolve these limitations, the Adaptive Moment Estimation (Adam) optimizer [20] computes individual, adaptive learning rates for each parameter by tracking exponential moving averages of both past gradients μ_t and past squared gradients ν_t [8, Sec. 4.5.5.2]:

$$\mu_t = \beta_1 \mu_{t-1} + (1 - \beta_1) \nabla_\theta \mathcal{L}(\mathcal{B}_t; \theta_{t-1}) \tag{3.9a}$$

$$\nu_t = \beta_2 \nu_{t-1} + (1 - \beta_2) \left(\nabla_\theta \mathcal{L}(\mathcal{B}_t; \theta_{t-1})\right)^2, \tag{3.9b}$$

where $\beta_1, \beta_2 \in [0, 1)$ are the exponential decay rates, and the square denotes the element-wise square of the current mini-batch gradient $\nabla_\theta \mathcal{L}(\mathcal{B}_t; \theta_{t-1})$.

Since μ_0 and ν_0 are initialized as zero vectors, both moments are biased towards zero. This is counteracted by dynamic correction factors $\hat{\mu}_t$ and $\hat{\nu}_t$, which are applied at each timestep t:

$$\hat{\mu}_t = \frac{\mu_t}{1 - \beta_1^t}, \qquad \hat{\nu}_t = \frac{\nu_t}{1 - \beta_2^t}. \tag{3.10}$$

The network parameters are then updated element-wise according to:

$$\theta_t = \theta_{t-1} - \eta \cdot \frac{\hat{\mu}_t}{\sqrt{\hat{\nu}_t} + \epsilon}, \tag{3.11}$$

where $\epsilon > 0$ is a small constant used for numerical stability. Following [20], the default hyperparameters are chosen as $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$, providing robust convergence across various deep learning architectures.

3.2 Adversarial Examples and Projected Gradient Methods

Deep neural networks are known to be prone to small, intentionally crafted perturbations that significantly change the model outputs or worsen the loss metrics [10], [11]. This vulnerability arises from the difference between the generalization of the average- and worst-case scenario. In this context, random sampling of inputs associated with high loss values may not be reliably detected, whereas gradient-based optimization actively searches for such directions[8, Sec. 12.4]. By applying gradient ascent to the inputs under the constraint of a bounded ℓ_p-norm $\|\delta\|_p \leq \varepsilon_\delta$, the approach targets regions where the objective value is large. In neural control, adversarial search strategies can be adapted to serve as tools for empirical falsification. The goal here is not to deceive a model, but rather to identify input configurations that identify violations of safety or stability requirements. This provides a computationally efficient approach to find high violations during training [7].

In supervised learning, the generation of adversarial examples can be formulated as a constrained optimization problem [10], [21]. Consider a trained model parameterized by θ, a nominal input $x \in \mathbb{R}^{n_0}$, and its target value $y \in \mathbb{R}^{n_L}$. Under a white-box assumption, the adversary has full knowledge of the network architecture and weights, which allows the computation of accurate input gradients $\nabla_x \mathcal{J}$. The objective is to find an optimal perturbation vector $\delta \in \mathbb{R}^{n_0}$ that maximizes the loss function $\mathcal{J}(x+\delta, y; \theta)$ [21]:

$$\max_{\delta} \quad \mathcal{J}(x+\delta, y; \theta) \qquad \text{s.t.} \quad \|\delta\|_p \leq \varepsilon_\delta, \tag{3.12}$$

where $\varepsilon_\delta > 0$ bounds the perturbation size under an ℓ_p-norm, with typical choices being $p \in \{2, \infty\}$ [10], [21]. In practical applications, the perturbed input may additionally be clipped to the valid input range.

3.2.1 Projected Gradient Descent

To solve the maximization problem (3.12), the Fast Gradient Sign Method (FGSM) [10] provides a fast, linear one-step approximation by taking a single step along the sign of the loss gradient. However, in highly non-convex loss landscapes, one-step attacks may yield weaker candidates than iterative attacks [21].

Projected Gradient Descent (PGD) extends FGSM into an iterative multi-step gradient ascent scheme. Following the robust optimization framework proposed by Madry [21], $\mathcal{P}$ denotes the set of allowed perturbations. For an ℓ_∞-norm constraint, $\mathcal{P}$ forms a hyper-rectangular box centered at the origin:

$$\mathcal{P} := \{\delta \in \mathbb{R}^{n_0} \mid \|\delta\|_\infty \leq \varepsilon_\delta\}. \tag{3.13}$$

Instead of initializing the search at the clean input x, the initial candidate $x_{\text{adv}}^{(0)}$ is sampled uniformly from $x + \mathcal{P}$ to prevent the optimization from getting stuck in suboptimal local maxima [21]. At each step $j \in \{0, \ldots, I-1\}$, the adversarial sample is updated along the gradient direction:

$$x_{\text{adv}}^{(j+1)} \leftarrow \Pi_{x+\mathcal{P}} \left(x_{\text{adv}}^{(j)} + \eta_{\text{PGD}} \operatorname{sign} \left(\nabla_{x_{\text{adv}}^{(j)}} \mathcal{J} \left(x_{\text{adv}}^{(j)}, y; \theta \right) \right) \right), \tag{3.14}$$

where $\eta_{\text{PGD}} > 0$ denotes the step size and $I \in \mathbb{N}$ represents the total number of iterations. Furthermore, $\Pi_{x+\mathcal{P}}(\cdot)$ projects the updated sample back onto the constraint set [21]. This projection can be implemented by clipping each input component to the interval $[x - \varepsilon_\delta, x + \varepsilon_\delta]$. As a result, $x_{\text{adv}}^{(j+1)}$ satisfies the perturbation constraint at every iteration.

3.2.2 Adaptation to Neural Lyapunov Control

When moving from conventional adversarial attacks to neural Lyapunov control, the adversary's objective changes. Rather than deceiving a classifier, the goal becomes to identify counterexample states x_{adv} that violate control-theoretic guarantees, such as Lyapunov decrease (2.3) or domain invariance (2.7c) [7].

Instead of maximizing a classification loss [10], [21], the gradient-based search is utilized to identify states associated with large violations of the Lyapunov conditions. In the setting further defined in Section 5.1 and Section 5.2.1, the Lyapunov candidate and the controller are parameterized independently as $V_\phi(x)$ and $\pi_\theta(x)$, respectively. Adapting the robust optimization paradigm [21] to deterministic control, their joint training can be formulated as a saddle-point problem:

$$\min_{(\phi,\theta)} \quad \max_{x_{\text{adv}} \in \mathcal{X}} \quad \mathcal{J}(x_{\text{adv}}; \phi, \theta) \tag{3.15}$$

Here, the inner maximization searches for candidate states within the training domain $\mathcal{X}$ that yield large violations of the Lyapunov conditions. The objective $\mathcal{J}(x_{\text{adv}}; \phi, \theta)$ therefore depends on both the Lyapunov-function parameters ϕ and the controller parameters θ. Conversely, the parameters are updated during the outer optimization, to resolve these violations [21].

Since uniform sampling may fail to discover violations in high-dimensional spaces[8, Sec. 12.4], gradient-guided ascent along $\nabla_x \mathcal{J}$ can focus the search on candidate states associated with large Lyapunov condition violations. In this context, PGD serves as an adversarial training mechanism, iteratively feeding high-violation candidate states back into the optimization to reinforce Lyapunov stability margins [7], [21].

4 Formal Verification of Neural Networks

When using neural networks as control policies in safety-critical domains, it is essential to provide formal guarantees of safety, robustness, and stability for all states within the region of interest. Verification methods for neural networks are generally categorized into two approaches: *Incomplete verifiers* efficiently compute sound lower bounds using convex relaxations, but may yield inconclusive results if the over-approximation gap is too large. In contrast, *complete verifiers* provide a definite decision by reducing this over-approximation gap, ensuring that the prescribed condition is either formally proven or a concrete counterexample is found.

Early complete verifiers typically used exact non-convex activation functions with specialized Satisfiability Modulo Theories (SMT) [13] or Mixed-Integer Linear Programming (MILP) [14] solvers. Although these solvers guarantee completeness, their complexity restricts their scalability on larger networks, which has led to the development of branch-and-bound (BaB) [15] frameworks that utilize parallelized incomplete bounds.

To perform scalable formal verification, this work relies on the α,β-CROWN framework [16], [17], which utilizes GPU parallelization and efficient bound propagation. This chapter covers the theoretical foundations of the verification methods used in α,β-CROWN: Section 4.1 introduces incomplete verification based on linear bound propagation via CROWN [9] and its optimized variant α-CROWN [16]. The subsequent Section 4.2 discusses complete verification using branch-and-bound in combination with β-CROWN [17] to soundly verify properties subject to neuron split constraints.

4.1 Incomplete Verification

The aim of formal neural network verification is to prove that a prescribed condition holds for all inputs within a predefined domain $\mathcal{F} = \{x \in \mathbb{R}^{n_0} \mid \underline{x}_F \leq x \leq \overline{x}_F\}$. Consider an L-layer feed-forward neural network as defined in (3.1), along with a scalar specification function $\psi(a^{(L)}) \in \mathbb{R}$, where $\psi(a^{(L)}) > 0$ denotes satisfaction of the safety or stability condition. The verification problem is formulated as a constrained global minimization problem:

$$\psi^\star = \min_{x \in \mathcal{F}} \psi(a^{(L)}) \quad \text{s.t.} \quad z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \sigma^{(l)}\left(z^{(l)}\right), \quad a^{(0)} = x, \tag{4.1}$$

for all layers $l \in \{1, \ldots, L\}$. Note that the network parameters $\{W^{(l)}, b^{(l)}\}_{l=1}^{L}$ are fixed during verification. Hence, for every input $x \in \mathcal{F}$, all intermediate variables $z^{(l)}(x)$ and $a^{(l)}(x)$ are uniquely determined by the forward pass.

The given condition is formally satisfied over the whole domain $\mathcal{F} \subset \mathbb{R}^{n_0}$ if and only if the global minimum satisfies $\psi^\star > 0$. Nonlinear activation functions result in nonconvexity, which makes it

computationally difficult to determine the exact minimum $\psi^\star$. In order to circumvent this computational limitation, incomplete verification methods approximate the nonlinearities by convex relaxations. These incomplete verification methods yield a guaranteed lower bound $\underline{\psi} \leq \psi^\star$, where:

- **Verified if $\underline{\psi} > 0$:** Since $0 < \underline{\psi} \leq \psi^\star \implies \psi^\star > 0$, the specification $\psi(a^{(L)}(x)) > 0$ is formally certified for all $x \in \mathcal{F}$, ensuring soundness.

- **Inconclusive if $\underline{\psi} \leq 0$:** No conclusion can be drawn. It remains unknown whether there exists an actual counterexample or whether the relaxations are too loose.

Consequently, incomplete verifiers are computationally efficient and scalable, enabling both fast standalone certification and efficient bounding in complete verification frameworks.

4.1.1 CROWN

While traditional incomplete verifiers rely on computationally expensive linear programs (LPs) solved over the entire network, the CROWN framework [9] achieves significantly higher efficiency with backward linear bound propagation. To construct linear relaxations, intermediate pre-activation bounds $[l_j^{(l)}, u_j^{(l)}] \in \mathbb{R}$ are required for each neuron j in layer l, which are computed by using pre-activations as temporary verification objectives within the bound propagation framework. Given these intermediate intervals, CROWN upper- and lower-bounds a wide range of continuous or piecewise-linear activation function $\sigma(z_j^{(l)})$ over their respective intervals by linear bounding functions:

$$\alpha_{L,j}^{(l)} z_j^{(l)} + d_{L,j}^{(l)} \;\leq\; \sigma\left(z_j^{(l)}\right) \;\leq\; \alpha_{U,j}^{(l)} z_j^{(l)} + d_{U,j}^{(l)}, \tag{4.2}$$

where $\alpha_{L,j}^{(l)}, \alpha_{U,j}^{(l)} \in \mathbb{R}$ denote the slopes and $d_{L,j}^{(l)}, d_{U,j}^{(l)} \in \mathbb{R}$ denote the intercepts of the bounding lines.

For general nonlinear activations with a differentiable curvature, such as tanh or sigmoid, CROWN divides the neurons of layer l into three distinct subsets based on the signs of their pre-activation bounds $[l_j^{(l)}, u_j^{(l)}]$, as shown in Fig. 4.1:

- **Strictly Concave Region ($\zeta_l^+ = \{j \mid 0 \leq l_j^{(l)} \leq u_j^{(l)}\}$):** The upper bound is chosen as a tangent at an adaptive point $z^\star \in [l_j^{(l)}, u_j^{(l)}]$, while the lower bound is the secant line connecting the interval endpoints $(l_j^{(l)}, \sigma(l_j^{(l)}))$ and $(u_j^{(l)}, \sigma(u_j^{(l)}))$.

- **Strictly Convex Region ($\zeta_l^- = \{j \mid l_j^{(l)} \leq u_j^{(l)} \leq 0\}$):** The upper bound is the secant connecting the interval endpoints, whereas the lower bound is formed by a tangent at an adaptive point $z^\star \in [l_j^{(l)}, u_j^{(l)}]$.

- **Mixed Curvature with Inflection Point ($\zeta_l^\pm = \{j \mid l_j^{(l)} < 0 < u_j^{(l)}\}$):** The upper bound is the tangent passing through the left boundary $(l_j^{(l)}, \sigma(l_j^{(l)}))$ with contact point $z^\star \geq 0$, and the lower bound is the tangent passing through the right boundary $(u_j^{(l)}, \sigma(u_j^{(l)}))$ with contact point $z^\star \leq 0$.

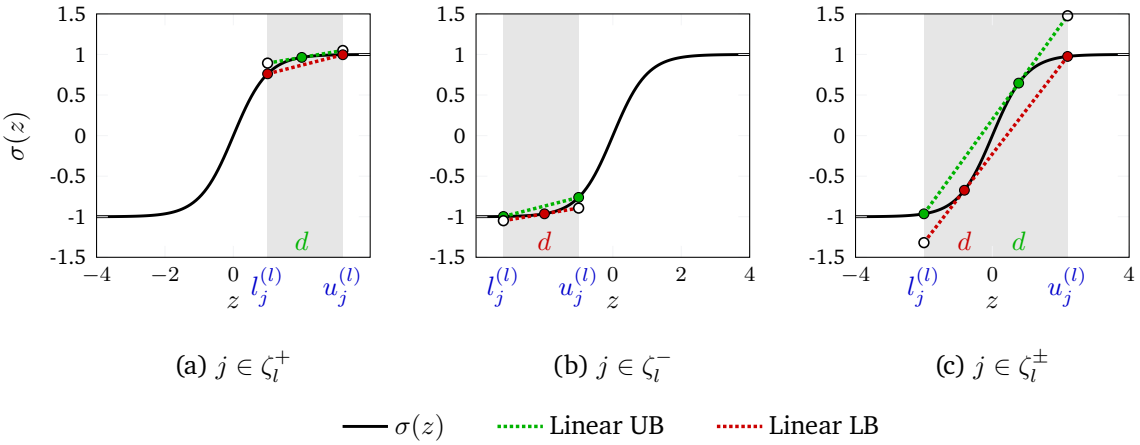

Figure 4.1: Linear bounds for general S-shaped activation functions $\sigma(z) = \tanh(z)$, inspired by Zhang et al. [9]. The green dotted lines denote the upper bounds and the red dotted lines denote the lower bounds.

Piecewise-linear activations such as ReLU represent a special case within this framework [9]. Stable neurons in ζ_l^+ and ζ_l^- are evaluated exactly without relaxation, yielding $\sigma(z_j^{(l)}) = z_j^{(l)}$ and $\sigma(z_j^{(l)}) = 0$, respectively. Only unstable neurons in $\zeta_l^\pm$ are relaxed using the triangle relaxation:

$$\alpha_{L,j}^{(l)} z_j^{(l)} \;\leq\; \left[z_j^{(l)}\right]_+ \;\leq\; \frac{u_j^{(l)}}{u_j^{(l)} - l_j^{(l)}} \left(z_j^{(l)} - l_j^{(l)}\right), \tag{4.3}$$

where $\alpha_{L,j}^{(l)} \in [0, 1]$ parameterizes the slope of the linear lower bound.

By recursively propagating these linear bounds backward to the network input, CROWN constructs an affine function

$$\underline{\psi}_{\text{aff}}(x) := w_{\text{CROWN}}^\top x + c_{\text{CROWN}} \leq \psi(a^{(L)}(x)) \qquad \forall x \in \mathcal{F}. \tag{4.4}$$

Here, $w_{\text{CROWN}} \in \mathbb{R}^{n_0}$ and $c_{\text{CROWN}} \in \mathbb{R}$ are the coefficients of the resulting input-dependent affine lower bound. The certified lower bound is obtained by minimizing this affine surrogate over the input space:

$$\underline{\psi}^\star_{\text{CROWN}} = \min_{x \in \mathcal{F}} \underline{\psi}_{\text{aff}}(x). \tag{4.5}$$

4.1.2 α-CROWN

In the CROWN formulation, the lower bound slope $\alpha_{L,j}^{(l)}$ in (4.2) is selected based on fixed heuristic rules [9]. For instance, for unstable ReLU activations, standard heuristics assign $\alpha_{L,j}^{(l)} = 1$ if $u_j^{(l)} > |l_j^{(l)}|$ and $\alpha_{L,j}^{(l)} = 0$ otherwise. For smooth activations, CROWN analogously selects activation-specific local linear bounds based on the corresponding pre-activation interval. While computationally efficient, fixed rules often result in loose lower bounds, which may degrade verification tightness. α-CROWN [16] addresses this limitation by considering relaxation parameters as continuous optimizable parameters over all layers. For ReLU activations, these correspond to the lower-bound slopes $\alpha \in [0, 1]^{N_{\text{unstable}}}$ of all

unstable neurons, where $N_{\text{unstable}} = \sum_{l=1}^{L-1} |\zeta_l^{\pm}|$ denotes the total number of unstable neurons in the network. More generally, this principle applies to any adjustable parameter defining valid linear bounds for general activations. Consequently, the bounding coefficients become functions of α, yielding the parameterized lower-bound objective:

$$\mathcal{G}(\alpha) = \min_{x \in \mathcal{F}} \left(w(\alpha)^\top x + c(\alpha) \right), \tag{4.6}$$

where $w(\alpha) \in \mathbb{R}^{n_0}$ and $c(\alpha) \in \mathbb{R}$ denote the affine lower bound coefficients, respectively. The verification problem is then formulated as a maximization of this lower bound:

$$\underline{\psi}_\alpha^\star = \max_{\alpha \in [0,1]^K} \mathcal{G}(\alpha) \;\leq\; \psi^\star. \tag{4.7}$$

The objective $\mathcal{G}(\alpha)$ can be optimized efficiently on GPUs using projected gradient-based methods, such as projected Adam [20]. Crucially, every intermediate configuration $\alpha \in [0,1]^K$ defines a valid relaxation. Therefore, early stopping inherently ensures soundness while also providing potentially tighter bounds than static heuristics.

4.2 Complete Verification

When incomplete methods yield an inconclusive result, complete verifiers are required to obtain a definitive decision. These verifiers typically employ a Branch-and-Bound (BaB) framework [15], [17]. Conceptually, BaB resolves the nonlinearities by systematically splitting the search space, either by branching over the input domain or by splitting unstable intermediate neurons. Within each branch of the search tree, an incomplete verifier evaluates the lower bound $\underline{\psi}_{\text{sub}}$:

- **Pruning:** Subdomains that satisfy $\underline{\psi}_{\text{sub}} > 0$ are formally certified and discarded from further search.
- **Branching / Falsification:** Inconclusive subdomains with $\underline{\psi}_{\text{sub}} \leq 0$ are partitioned further, or evaluated to find a concrete counterexample.

By iteratively adding branching constraints, the approximation error is systematically reduced across all subdomains. For piecewise-linear neural networks, this guarantees that complete verification terminates in a finite number of branching steps to yield a definitive decision [15], [17].

4.2.1 α,β-CROWN

When applying Branch-and-Bound to neural networks with piecewise-linear activations such as ReLU, each branching step results in split constraints for unstable neurons. Specifically, an unstable neuron j in layer l is either forced into its active linear phase $z_j^{(l)} \geq 0$ or into its inactive phase $z_j^{(l)} < 0$ [17]. While classical LP solvers handle these constraints natively, they are computationally expensive and cannot be efficiently parallelized on GPUs [15]. However, unlike LP solvers, standard linear bound propagation frameworks such as CROWN cannot incorporate constraints on intermediate neurons, yielding loose bounds and redundant branching [16], [17].

To overcome this limitation, β-CROWN [17] incorporates neuron split constraints directly into backward bound propagation using Lagrangian relaxation. By assigning non-negative Lagrange multipliers $\beta \geq 0$

to each split condition, constraint violations are penalized linearly during the backward pass. The combination of the split constraints with the relaxation slopes $\alpha \in [0,1]^{N_{\text{unstable}}}$ from α-CROWN yields the unified α,β-CROWN framework. Propagating both relaxation parameters back to the input layer constructs the linear lower-bounding surrogate:

$$w(\alpha,\beta)^\top x + c(\alpha,\beta) \;\leq\; \psi(a^{(L)}(x)), \tag{4.8}$$

where $w(\alpha,\beta) \in \mathbb{R}^{n_0}$ and $c(\alpha,\beta) \in \mathbb{R}$ denote the resulting surrogate coefficients. Minimizing this surrogate over the input domain $\mathcal{F}_{\text{sub}} \subseteq \mathcal{F}$ of a given BaB subproblem defines the parameterized lower-bound objective:

$$\mathcal{G}(\alpha,\beta) := \min_{x\in\mathcal{F}_{\text{sub}}} \Big(w(\alpha,\beta)^\top x + c(\alpha,\beta) \Big). \tag{4.9}$$

For each BaB subdomain, the tightest lower bound is determined by jointly maximizing $\mathcal{G}(\alpha,\beta)$ over the relaxation slopes α and Lagrange multipliers β:

$$\underline{\psi}^\star_{\alpha,\beta} = \max_{\alpha\in[0,1]^{N_{\text{unstable}}},\,\beta\geq 0} \mathcal{G}(\alpha,\beta) \;\leq\; \psi^\star_{\text{sub}}. \tag{4.10}$$

Although $\mathcal{G}(\alpha,\beta)$ is concave w.r.t. β for fixed linear relaxations, jointly optimizing over both the relaxation slopes α and the Lagrange multipliers β leads to a non-concave optimization problem. Nonetheless, projected gradient-based optimizers can be effectively applied on GPUs to tighten the bounds. Because any feasible configuration $\big(\alpha \in [0,1]^{N_{\text{unstable}}}, \beta \geq 0\big)$ corresponds to a valid dual relaxation, soundness is unconditionally preserved under early stopping, allowing fast and constraint-aware bounding in parallel.

Beyond branching over activation spaces, α,β-CROWN can also branch directly over the input domain. This input domain splitting is often more efficient for neural control systems with low-dimensional continuous state spaces. Furthermore, gradient-based adversarial attacks, such as PGD, may also be applied prior to the Branch-and-Bound search to identify potential counterexamples. If a counterexample is found, the Branch-and-Bound search is not needed and the verification terminates immediately.

In summary, the verification methods presented in this chapter provide a flexible range of trade-offs between computational speed and bounding precision. CROWN offers efficient incomplete verification and can therefore be utilized in the Lyapunov co-training described in Section 5.2, where its lower bounds are included in the LiRPA-condition loss. In contrast, complete α,β-CROWN verification provides formal certification by refining the verification domain. In the methodology presented in the following chapter, both approaches are combined: CROWN is used during training to promote a verifier-friendly Lyapunov formulation, while α,β-CROWN is subsequently applied to formally certify the maximal region of attraction $\mathcal{V}_{\rho^\star}$.

5 Learning Lyapunov-Stable Imitation Controllers

This chapter presents the methodology for learning and certifying a neural approximation of the MPC controller. The MPC policy introduced in Section 2.2.1 serves as an expert, generating state-input pairs for supervised policy training. As described in Section 5.1, this allows the neural policy to reproduce the expert control actions over the considered state domain. Subsequently, a neural Lyapunov candidate is learned to enable the stability certification of the controller. In the co-training setting, the policy parameters may additionally be refined, while the Lyapunov architecture and training objective enforce the stability conditions detailed in Section 5.2. Finally, to formally prove that the learned controller satisfies these guarantees across the continuous state space, the networks are subjected to formal verification using α,β-CROWN [16], [17], as presented in Section 5.3.

5.1 Neural Policy Approximation and Imitation Learning

Data Generation uses the MPC expert policy π_0 defined in Section 2.2.3. To ensure solver convergence and obtain consistent expert trajectories during data generation, the shifted initialization is used[1, Sec. 12.5]. By shifting the previously computed optimal sequence $(u^\star_{i|k})_{i=0}^{N-1}$ and appending the local terminal controller $\pi_f(x^\star_{N|k})$, the optimization problem initializes from a feasible candidate trajectory. This prevents the solver from getting stuck in local minima between adjacent sampling instances. Closed-loop trajectories $x_{k+1} = f(x_k, \pi_0(x_k))$ are initialized from states sampled across the mpc sampling set $\mathcal{X}_{\text{MPC}} := \{x \in \mathcal{X} \mid \underline{x}_{\text{MPC}} \leq x \leq \overline{x}_{\text{MPC}}\}$. However, the training set is defined as $\mathcal{X}_\pi := \{x \in \mathcal{X} \mid \underline{x}_\pi \leq x \leq \overline{x}_\pi\}$. Storing the visited states and their corresponding optimal control actions yields the dataset

$$\mathcal{D}_\pi := \left\{(x^{(i)}, u^{(i)})\right\}_{i=1}^{N_\mathcal{D}}, \tag{5.1}$$

where $x^{(i)} \in \mathcal{X}_\pi$ and $u^{(i)} = \pi_0(x^{(i)})$.

Imitation Policy follows the structure proposed in [7], as it enforces true control inputs at the equilibrium $\pi_\theta(x^\star) = u^\star$ and satisfies the control input constraints $\underline{u} \leq \pi_\theta(x) \leq \overline{u}$ by construction. The unconstrained neural policy $\tilde{\pi}_\theta$ is formulated as

$$\tilde{\pi}_\theta(x) = h_\theta(x) - h_\theta(x^\star) + u^\star. \tag{5.2}$$

where $h_\theta : \mathbb{R}^{n_x} \to \mathbb{R}^{n_u}$ is a neural network and $u^\star \in \mathcal{U}$ represents the desired control action at the equilibrium $x^\star$. However, to ensure the satisfaction of the control input constraints, the unconstrained

output $\tilde{\pi}_\theta(x)$ is clipped element-wise using $\text{clamp}(\cdot, \underline{\cdot}, \overline{\cdot})$. This yields the neural policy visualized in Fig. 5.1:

$$\pi_\theta(x) = \text{clamp}(\tilde{\pi}_\theta(x), \underline{u}, \overline{u}). \tag{5.3}$$

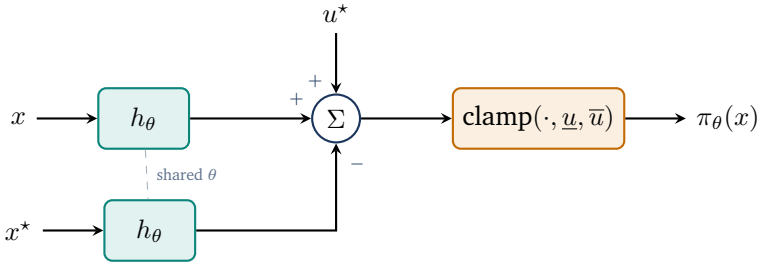

Figure 5.1: The imitation learning policy.

Imitation Objective is a multi-objective loss, which consists of two parts: a state-weighted behavioral cloning loss and a dynamics-aware penalty. To account for different physical units and numerical scales across the state and control vectors, the loss terms are normalized using strictly positive diagonal scaling matrices

$$S_x = \text{diag}(x_{\text{scale}}) \in \mathbb{R}^{n_x \times n_x} \quad \text{and} \quad S_u = \text{diag}(u_{\text{scale}}) \in \mathbb{R}^{n_u \times n_u}. \tag{5.4}$$

Using S_x, the normalized distance of a state x from the equilibrium $x^\star$ is quantified with the ℓ_1-norm as

$$d(x) = \frac{1}{n_x} \left\| S_x^{-1}(x - x^\star) \right\|_1. \tag{5.5}$$

To assign appropriate priority during behavioral cloning, a state-dependent weight is defined as

$$w_x(x) = w_{\min} + (1 - w_{\min}) \left(\frac{1}{1 + d(x)} \right)^{\gamma_x}, \tag{5.6}$$

where $w_{\min} \in (0, 1)$ is the baseline weight for states far from the equilibrium and $\gamma_x \geq 0$ controls the decay rate.

State-Weighted Behavioral Cloning Loss penalizes the squared normalized Euclidean distance between the neural network policy and the expert targets. For a mini-batch $\mathcal{B}_\pi \subset \mathcal{D}_\pi$ of size $|\mathcal{B}_\pi|$, the behavioral cloning loss is defined as

$$\mathcal{L}_{\text{BC}} = \frac{1}{|\mathcal{B}_\pi| n_u} \sum_{(x,u) \in \mathcal{B}_\pi} w_x(x) \left\| S_u^{-1} \big(\pi_\theta(x) - u \big) \right\|_2^2. \tag{5.7}$$

Here, the state-dependent weight $w_x(x)$ prioritizes approximation accuracy near the equilibrium.

Dynamics-Aware Constraint Loss applies a one-step penalty for successor states that leave the admissible state set $\mathcal{X}_\pi$. To maintain non-zero gradient flow even in regions of the state space where the control input saturates, the one-step forward dynamics are evaluated for all $x \in \mathcal{B}_\pi$ using the unconstrained neural policy as $x^+ = f(x,\tilde{\pi}_\theta(x))$. Using the element-wise ReLU function defined in (3.2b), the loss is formulated as

$$\mathcal{L}_f = \frac{1}{|\mathcal{B}_\pi| n_x} \sum_{x \in \mathcal{B}_\pi} \left(\left\| \left[x^+ - \overline{x}_\pi \right]_+ \right\|_1 + \left\| \left[\underline{x}_\pi - x^+ \right]_+ \right\|_1 \right). \tag{5.8}$$

Here, a one-step evaluation outside the admissible set results in a penalty for the current state, forcing the learned policy to stay inside the admissible set.

Total Imitation Loss is the combination of the state-weighted behavioral cloning and dynamics-aware constraint losses resulting in a scalar objective. Scaling each term by its respective weight, λ_{BC} and λ_f, yields the overall policy loss:

$$\mathcal{L}_\pi = \lambda_{\text{BC}} \mathcal{L}_{\text{BC}} + \lambda_f \mathcal{L}_f. \tag{5.9}$$

By minimizing $\mathcal{L}_\pi$, the policy π_θ learns to mimic the expert policy π_0, while penalizing state constraint violations in $\mathcal{X}_\pi$. In combination with the policy architecture in (5.3), π_θ satisfies the input constraints $\underline{u} \leq \pi_\theta(x) \leq \overline{u}$ for all states $x \in \mathbb{R}^{n_x}$ and guarantees exact equilibrium control $\pi_\theta(x^\star) = u^\star$. Additionally, the state-dependent weighting in (5.6) prioritizes approximation accuracy near the equilibrium, which may be beneficial for establishing Lyapunov stability. Now, this imitation learning pipeline establishes an accurate expert imitation, serving as an initial policy for the subsequent Lyapunov co-training.

5.2 Lyapunov Learning for Stability Certification

To obtain a controller π_θ and a Lyapunov candidate V_ϕ suitable for formal verification, a co-training scheme is utilized to train both models jointly. Under this optimization, the controller learns to satisfy the Lyapunov conditions while maintaining near-baseline performance through regularization against the initial imitation policy θ_0, alongside a Frobenius norm penalty on the candidate's linear term R_ϕ to prevent its collapse. To address localized stability violations and shape a verifier-friendly loss landscape, the framework combines an adversarial falsification loop that iteratively mines counterexamples with bound propagation losses, while continuously expanding the estimated region of attraction $\mathcal{V}_\rho$. Each of the training components are described in the following subsections.

5.2.1 Lyapunov Network Architecture

Following [7], the Lyapunov candidate $V_\phi(x)$ is parameterized to be positive definite by construction:

$$V_\phi(x) = |g_\phi(x) - g_\phi(x^\star)| + \left\| (\varepsilon I + R_\phi^\top R_\phi)(x - x^\star) \right\|_1, \tag{5.10}$$

where $g_\phi : \mathbb{R}^{n_x} \to \mathbb{R}$ is a neural network, $x \in \mathbb{R}^{n_x}$ is the current state, $x^\star \in \mathbb{R}^{n_x}$ is the equilibrium, $\varepsilon > 0$ is a small regularization constant, and $R_\phi \in \mathbb{R}^{n_x \times n_x}$ is a learnable matrix. While the first term is only positive semi-definite and may vanish away from the equilibrium, the second term guarantees strict positive definiteness $\forall x \in \mathcal{X} \setminus \{x^\star\}$, as the matrix $(\varepsilon I + R_\phi^\top R_\phi)$ is strictly positive definite by construction for any $\varepsilon > 0$. The complete architectural layout of $V_\phi(x)$ is illustrated in Fig. 5.2.

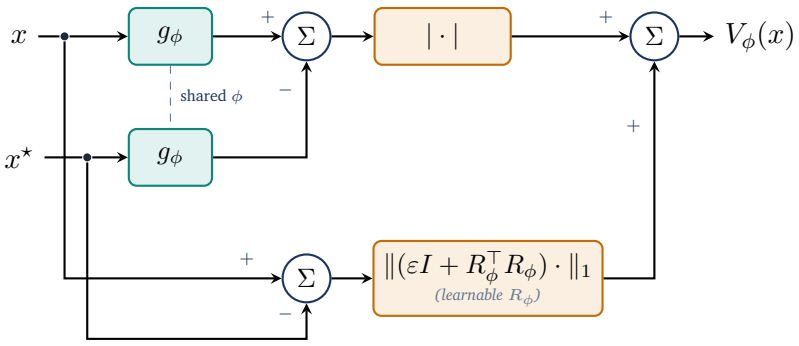

Figure 5.2: The neural Lyapunov architecture.

5.2.2 Loss Formulation

To ensure that the Lyapunov candidate described in (5.10) can be certified for stability, the loss function must comprise multiple components. It utilizes the previously introduced ReLU function $[\cdot]_+$ for one-sided penalties, alongside a weighted mean to aggregate the sample losses. Central to the following loss terms is the estimation of the region of attraction (ROA). Throughout the training process, a target level ρ is considered, leading to the corresponding ρ-sublevel set within the training domain $\mathcal{X}_V := \{x \in \mathcal{X}_\pi \mid \underline{x}_V \leq x \leq \overline{x}_V\}$, defined as

$$\mathcal{V}_\rho := \{x \in \mathcal{X}_V \mid V_\phi(x) \leq \rho\}\,. \tag{5.11}$$

To prevent this certified region from trivially collapsing towards the origin during optimization, and to ensure numerical stability during normalizations, ρ is strictly constrained by a lower bound $\rho_{\text{min}} > 0$. Furthermore, the successor states $x_V^+ = f(x, \pi_V(x))$ needed during loss computation are evaluated on-the-fly using the system dynamics and the policy. The latter may also be trained in the co-training setup and is thus referred to as π_V hereafter.

Relative Lyapunov Decrease Loss penalizes trajectories along which the Lyapunov candidate fails to decay. For each sample, the decrease violation is defined as

$$e_V = V_\phi(x_V^+) - (1-\kappa)V_\phi(x), \tag{5.12}$$

where $\kappa \in (0,1)$ is a hyperparameter that controls the desired rate of decrease. To balance the optimization scale across different magnitudes of V_ϕ, we normalize this violation to obtain the relative decrease loss:

$$\mathcal{L}_{\text{dec}} = \left[\frac{e_V}{\max(|V_\phi(x)|, \varepsilon_{\text{rel}})}\right]_+, \tag{5.13}$$

where $\varepsilon_{\text{rel}} > 0$ prevents division by zero. Note that the decrease normalization can be omitted depending on the training configuration.

Relative State Invariance Loss penalizes successor states leaving the Lyapunov training domain $\mathcal{X}_V$. To evaluate the relative one-step state-constraint violation, the scaling matrix $S_{\text{range}} = \text{diag}(\overline{x}_V - \underline{x}_V)$ is utilized. The loss per sample for the next state x_V^+ is computed as

$$\mathcal{L}_{\text{inv}} = \left\| S_{\text{range}}^{-1}\left[x_V^+ - \overline{x}_V\right]_+ \right\|_1 + \left\| S_{\text{range}}^{-1}\left[\underline{x}_V - x_V^+\right]_+ \right\|_1. \tag{5.14}$$

Combined ρ-Gated Condition Loss is the sample-wise ρ-gated combination of the relative Lyapunov decrease loss from (5.13) and the relative state invariance loss from (5.14),

$$\mathcal{L}_{\text{cond}} = \mathcal{L}_{\text{dec}} + \lambda_{\text{inv}} \cdot \mathcal{L}_{\text{inv}}, \tag{5.15}$$

where λ_{inv} is a hyperparameter balancing the relative importance of the two loss components. However, to apply the ρ-gate appropriately, $\mathcal{L}_{\text{cond}}$ is weighted for each sample using the soft sublevel weight

$$w_\rho = \sigma\left(s \cdot \frac{\rho - V_\phi(x)}{\max(\rho, \varepsilon_{\text{rel}})}\right), \tag{5.16}$$

where $s > 0$ denotes the gate sharpness. This smooth formulation ensures that the weights inside $\mathcal{V}_\rho$ are ≈ 1, whereas the weights for states far outside of ρ approach zero. Evaluating these terms on a mini-batch $\mathcal{B}_\rho$ drawn from the combined state pools $\mathcal{S} \cup \mathcal{C}$ (details in Section 5.2.3), the weighted mean of the combined condition loss results in the final loss formulation:

$$\mathcal{L}_{\rho-\text{gated}} = \frac{\sum_{\mathcal{B}_\rho} w_\rho \mathcal{L}_{\text{cond}}}{\sum_{\mathcal{B}_\rho} w_\rho}. \tag{5.17}$$

The complete data flow, from evaluating the batch constraints to computing this final gated loss, is summarized in Fig. 5.3.

ROA Loss tries to enlarge the region of attraction by uniformly drawing a batch $\mathcal{B}_{\text{ROA}}$ from $\mathcal{X}_V \setminus \xi\mathcal{X}_V$, where $\xi \in [0, 1)$ is a scaling factor defining the inner boundary of the shell, as shown in Fig. 5.4. The loss is then evaluated over these candidates using [7]

$$\mathcal{L}_{\text{ROA}} = \frac{1}{|\mathcal{B}_{\text{ROA}}|} \sum_{x \in \mathcal{B}_{\text{ROA}}} \ln\left(\left[\frac{V_\phi(x)}{\rho} - 1\right]_+ + 1\right). \tag{5.18}$$

Consequently, the loss function penalizes shell samples that fall outside the current ρ subsets. As the estimated region of attraction expands, fewer samples are penalized, causing the loss to decrease.

LIRPA-Condition Loss makes the Lyapunov function more verifier-friendly by penalizing formal violations of the Lyapunov decrease condition as defined in (5.12). These violations are identified using the LIRPA framework [16]. To reduce the computational cost during training, LIRPA is applied only to a heuristic subset of regions. Specifically, a hyper-rectangular cell is selected if at least one of its 2^{n_x} corners, or optionally its center point x_{center}, lies within the sublevel set $\mathcal{V}_\rho$. For each selected region, LIRPA then provides a formal lower bound $e_{V,\text{lower}}$ on the signed Lyapunov decrease $-e_V$. After evaluation, the LIRPA-condition violation for each region is computed as

$$\mathcal{M}_{\text{L}} = \frac{\ln([-e_{V,\text{lower}}]_+ + 1)}{\max(V_\phi(x_{\text{center}}), \varepsilon_{\text{rel}})}. \tag{5.19}$$

The final LIRPA-condition loss is then computed as the weighted mean over all evaluated regions, where the weight is defined using an exponential function of the squared Euclidean distance between the region center x_{center} and the equilibrium $x^\star$:

$$w_{\text{L}} = \exp\left(-\gamma_{\text{L}} \cdot \|x_{\text{center}} - x^\star\|_2^2\right), \tag{5.20}$$

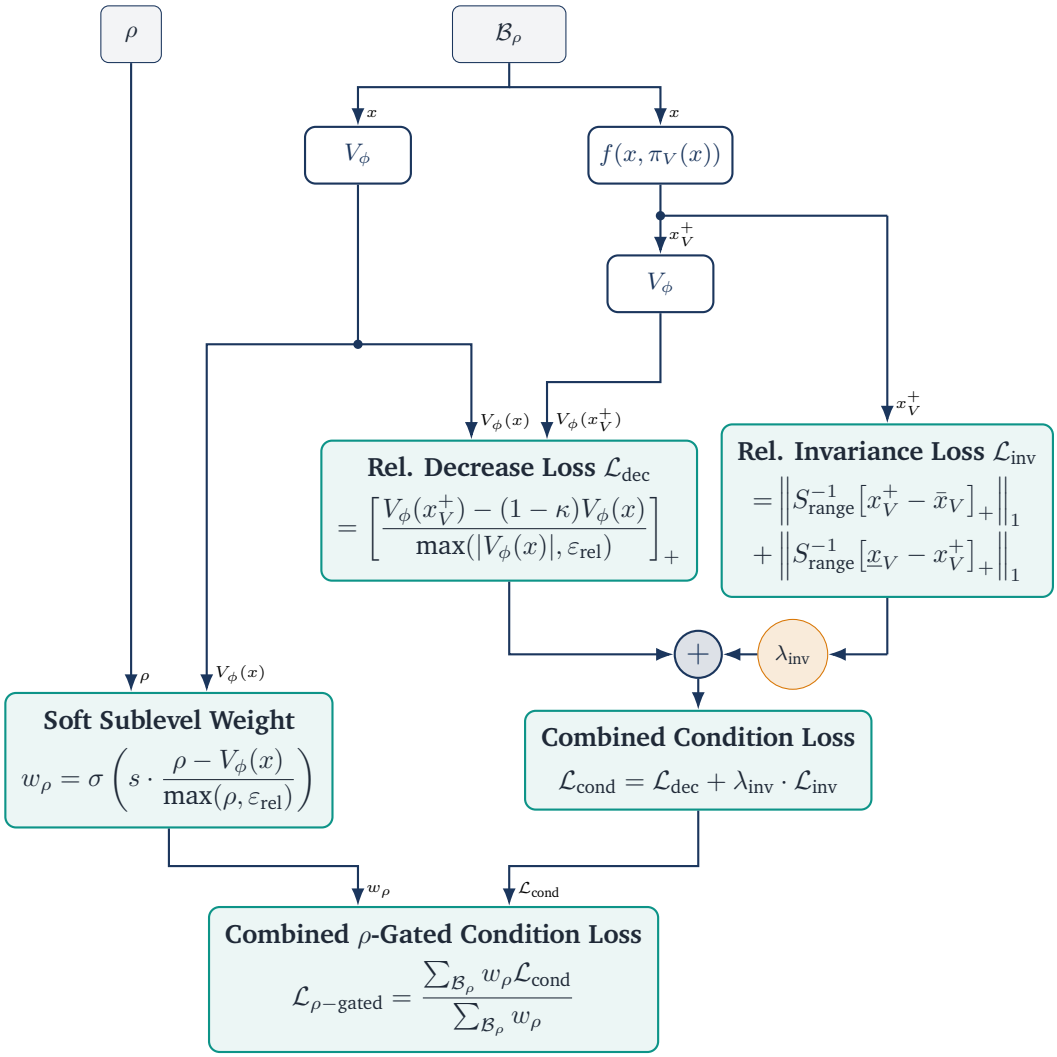

Figure 5.3: Combined gated condition loss $\mathcal{L}_{\rho-\text{gated}}$

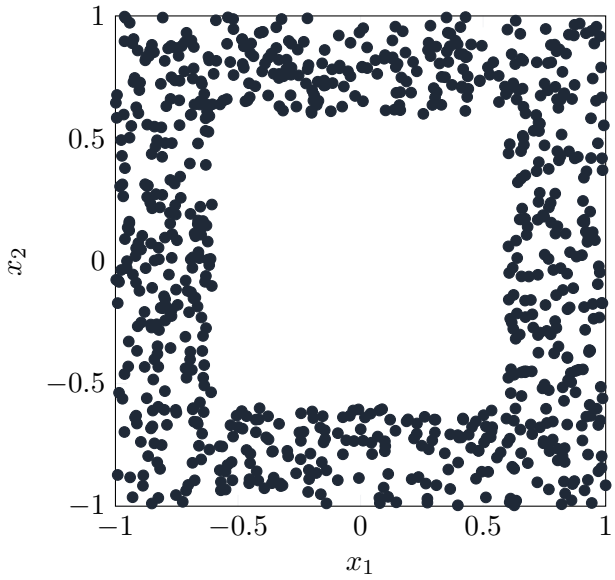

Figure 5.4: Illustration of uniformly drawn samples in the shell batch $\mathcal{B}_{\text{ROA}}$ with $\xi = 0.6$.

where $\gamma_{\text{L}} > 0$ is a hyperparameter controlling the decay rate of the weight. Finally, the LIRPA-condition loss over N_{L} regions is computed as

$$\mathcal{L}_{\text{L}} = \frac{\sum_{i=1}^{N_{\text{L}}} w_{\text{L}}^{(i)} \cdot \mathcal{M}_{\text{L}}^{(i)}}{\sum_{i=1}^{N_{\text{L}}} w_{\text{L}}^{(i)}}. \tag{5.21}$$

Lyapunov Scaling Loss prevents numerical collapse to zero by penalizing deviations from the initial average magnitude over a quasi-Monte Carlo batch $\mathcal{B}_V \subset \mathcal{X}_V$, sampled via a low-discrepancy Sobol sequence [22], as illustrated in Fig. 5.6. The logarithm of the mean Lyapunov value over this batch is defined as

$$\ell_V = \ln\left(\frac{1}{|\mathcal{B}_V|}\sum_{x\in\mathcal{B}_V} V_\phi(x)\right), \tag{5.22}$$

leading to the scaling loss formulation

$$\mathcal{L}_V = (\ell_{V,0} - \ell_V)^2. \tag{5.23}$$

The reference value $\ell_{V,0}$ is determined once using the initial Lyapunov candidate prior to training. To prevent the Lyapunov candidate from overfitting to a static point cloud, a new batch $\mathcal{B}_V$ is resampled from $\mathcal{X}_V$ with a specified probability at each optimization step.

Policy Scaling Loss encourages the learned policy π_V to remain close to the output of the original imitation controller π, which is based on the MPC teacher. A quasi-Monte Carlo batch $\mathcal{B}_{\text{pol}} \subset \mathcal{X}_V$ is sampled using a low-discrepancy Sobol sequence [22]. To prevent overfitting to a static set of states,

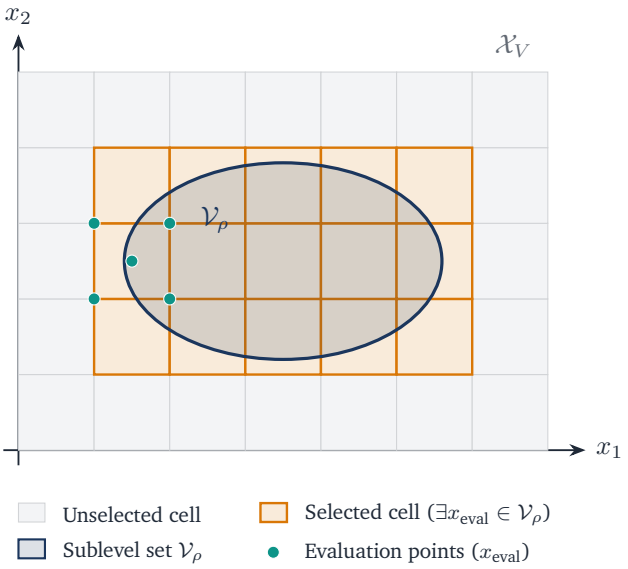

Figure 5.5: Lirpa regions selection with $\mathcal{V}_\rho$.

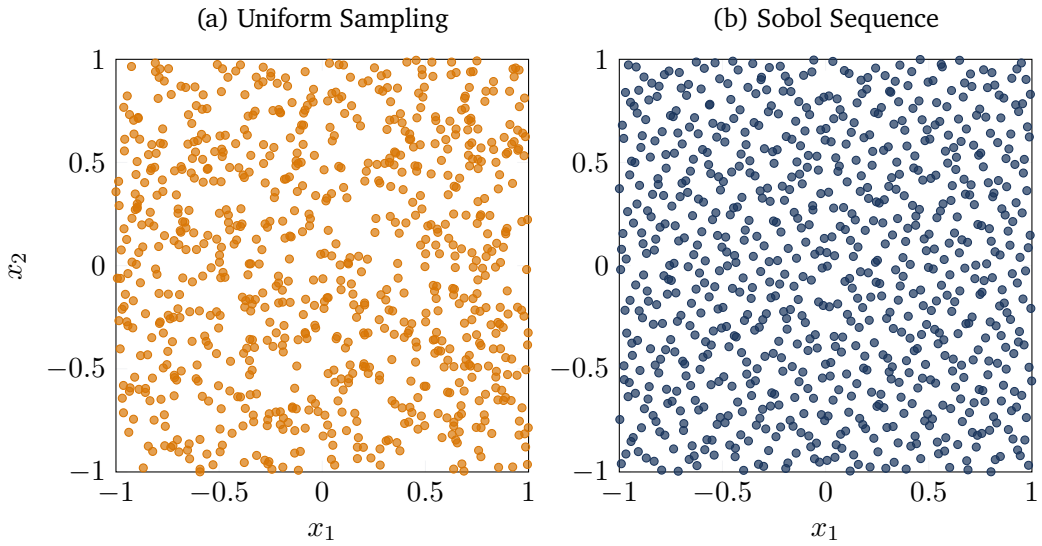

Figure 5.6: Comparison of Sobol and uniform sampling in two dimensions.

this batch is also resampled from $\mathcal{X}_V$ with a given probability at each optimization step. The normalized deviation between the learned policy and the imitation controller over this batch is formulated as

$$\mathcal{L}_{\text{pol}} = \frac{\sum_{x \in \mathcal{B}_{\text{pol}}} \|\pi_V(x) - \pi(x)\|_2^2}{\sum_{x \in \mathcal{B}_{\text{pol}}} \|\pi(x)\|_2^2} \tag{5.24}$$

R_ϕ-Matrix Loss regularizes the learnable matrix R_ϕ by penalizing deviations of its Frobenius norm from its initial value $\ell_{R_\phi,0}$:

$$\mathcal{L}_{R_\phi} = \left(\ell_{R_\phi,0} - \|R_\phi\|_F\right)^2. \tag{5.25}$$

However, $\ell_{R_\phi,0} = \|R_\phi\|_F$ is automatically computed, based on the initial R_ϕ matrix.

Overall Lyapunov Loss represents the combined objective function, summing all previously defined weighted components, given by

$$\mathcal{L}_{\text{total}} = \sum_m \lambda_m \mathcal{L}_m \tag{5.26}$$

where the weights $\lambda_m > 0$, with $m \in \{\rho-\text{gated}, \text{ROA}, \text{L}, V, \text{pol}, R_\phi\}$ are hyperparameters that balance the relative importance of the individual loss components.

5.2.3 Falsification-Guided Training

Relying solely on uniform state sampling is not sufficient to reliably detect violations of the Lyapunov condition, especially in higher-dimensional state spaces [7]. To overcome this problem, falsification-guided training actively searches for adversarial counterexamples and incorporates them into the training distribution as described in Section 3.2.2. This is made possible by an adversarial replay buffer that reconciles the exploration of the estimated attractor region with the targeted exploitation of known error modes.

PGD Falsification is utilized to identify counterexamples that violate the Lyapunov conditions and iteratively inject them into the counterexample pool $\mathcal{C}$. Specifically, an ℓ_∞-Projected Gradient Descent (PGD) is employed to search for adversarial states $x_{\text{adv}} \in \mathcal{X}_V$ that maximize the condition violation defined in (5.15) and (5.16). The per-sample PGD objective is defined as

$$\mathcal{J}(x; \phi, \theta) = w_\rho(x) \mathcal{L}_{\text{cond}}(x). \tag{5.27}$$

The adversarial search is initialized using the explorative pool $\mathcal{S}$, with up to 25% of the batch supplemented by existing counterexamples to encourage local refinement. Starting from an initial candidate $x_{\text{adv}}^{(0)}$, the algorithm maximizes $\mathcal{J}$ by iteratively updating candidate states along the sign of their loss gradient:

$$x_{\text{adv}}^{(j+1)} \leftarrow \Pi_{\mathcal{X}_V}\left(x_{\text{adv}}^{(j)} + \eta_{\text{PGD}} \operatorname{sign}\left(\nabla_{x_{\text{adv}}^{(j)}} \mathcal{J}\left(x_{\text{adv}}^{(j)}; \phi, \theta\right)\right)\right), \tag{5.28}$$

where $\eta_{\text{PGD}} > 0$ denotes the step size and $\Pi_{\mathcal{X}_V}(\cdot)$ projects the updated sample back onto the training domain bounds $\mathcal{X}_V$. To retain the strongest violations and mitigate the effects of overshooting, the state yielding the maximum objective value across all I iterations is preserved for each candidate. Ultimately, only states strictly within $\mathcal{V}_\rho$ that exceed the violation tolerance and lie outside the origin exclusion region are retained as counterexamples.

Adversarial Replay Buffer manages the sample distribution for the ρ-gated condition loss $\mathcal{L}_{\rho-\text{gated}}$. It maintains two distinct state pools: an explorative pool $\mathcal{S}$ and a counterexample pool $\mathcal{C}$.
As outlined in Algorithm 1, the explorative state pool $\mathcal{S}$ is repopulated at each outer epoch to balance global exploration with local refinement. First, candidate states $\mathcal{B}_\mathcal{S}$ are oversampled uniformly from the training domain $\mathcal{X}_V$. From these candidates, $N_\mathcal{S}$ states are probabilistically sampled based on a sigmoid weighting function centered around an expanded sublevel boundary $m_\rho \cdot \rho$ (with margin $m_\rho > 1$). This weighting smoothly penalizes states further outside the estimated region of attraction, significantly reducing their likelihood of being selected.

Algorithm 1: Probabilistic Explorative State Pool Resampling

Input: Lyapunov function V_ϕ, current ROA estimate ρ, margin m_ρ, sharpness s, target size $N_\mathcal{S}$
Output: Updated state pool $\mathcal{S}$ of size $N_\mathcal{S}$

1 Generate $4N_\mathcal{S}$ candidates: $\mathcal{B}_\mathcal{S} \sim \mathcal{U}(\mathcal{X}_V)$;

2 For all $x \in \mathcal{B}_\mathcal{S}$, assign weight $w(x) \leftarrow \sigma\left(s\,\frac{m_\rho \cdot \rho - V_\phi(x)}{\max(\rho, 10^{-9})}\right) + \varepsilon$;

3 $\mathcal{S} \leftarrow$ Sample $N_\mathcal{S}$ states from $\mathcal{B}_\mathcal{S}$ with probability $P(x) \propto w(x)$;

The counterexample pool $\mathcal{C}$ stores the critical violations identified during the PGD falsification step. An age-tracking mechanism limits sample lifespan, preventing overfitting to outdated violations. Each injected counterexample is assigned an initial age $\tau = 0$, which is incremented after every subsequent falsification phase. Once a counterexample reaches the maximum age τ_{max}, it is removed from the pool. Consequently, the condition loss continuously penalizes recent violations while discarding resolved edge cases as the Lyapunov candidate evolves.

During optimization, mini-batches $\mathcal{B}_\rho$ are sampled from the joint state distribution $\mathcal{S} \cup \mathcal{C}$, where the proportion of counterexamples $r_\mathcal{C}$ is dynamically adapted based on the number of newly mined violations $N_{\text{mined}} \in \mathbb{N}_0$. To smooth out fluctuations between mining steps, the empirical mining yield $r_{\text{mine}} = N_{\text{mined}}/N_\mathcal{S}$ is tracked using an exponential moving average (EMA):

$$\tilde{r}_{\text{mine}} \leftarrow \upsilon_\mathcal{C}\,\tilde{r}_{\text{mine}} + (1 - \upsilon_\mathcal{C})\,r_{\text{mine}}, \tag{5.29}$$

where $\upsilon_\mathcal{C} \in [0, 1)$ is the smoothing decay factor. The effective batch fraction $r_\mathcal{C}$ is then calculated using linear interpolation:

$$r_\mathcal{C} = r_{\text{min}} + \tilde{r}_{\text{mine}} \cdot (r_{\text{max}} - r_{\text{min}}), \tag{5.30}$$

where r_{min} and r_{max} denote the minimal and maximal possible counterexample proportion. Each mini-batch $\mathcal{B}_\rho$ is then formed by drawing $\lfloor r_\mathcal{C}\,|\mathcal{B}_\rho| \rfloor$ samples from the counterexample pool $\mathcal{C}$ and the remainder from $\mathcal{S}$.

5.2.4 ρ-Sublevel Estimation

To evaluate the ρ-gated condition loss in (5.17), the target sublevel bound ρ must be continuously estimated throughout training as the Lyapunov candidate $V_\phi(x)$ evolves. Since the goal is to identify the largest sublevel set $\mathcal{V}_\rho := \{x \in \mathcal{X}_V \mid V_\phi(x) \leq \rho\}$ that remains within the training domain $\mathcal{X}_V$, the sublevel set reaches the boundary of the domain at its maximum admissible value. Therefore, the theoretical upper bound for ρ is given by the minimum value of $V_\phi(x)$ on the boundary:

$$\rho_{\max} = \min_{x \in \partial \mathcal{X}_V} V_\phi(x). \tag{5.31}$$

Following the boundary-evaluation principle established by [7], evaluating $V_\phi(x)$ along the boundary $\partial \mathcal{X}_V$ is sufficient to estimate $\rho_{\max}$ without searching the entire domain. However, because $\partial \mathcal{X}_V$ can be a high-dimensional surface and $V_\phi(x)$ defines a non-convex landscape, simple uniform sampling on the boundary may fail to identify low-valued boundary points. Therefore, Projected Gradient Descent (PGD) is employed to search for local minima of $V_\phi(x)$ along the boundary.

Starting from uniform boundary seeds $\mathcal{B}_\partial \sim \mathcal{U}(\partial \mathcal{X}_V)$, PGD *iteratively minimizes* $V_\phi(x)$ along the negative gradient. To maintain candidate states on the hyper-rectangular boundary surface $\partial \mathcal{X}_V$, the projection operator $\Pi_{\partial \mathcal{X}_V}$ projects the updated states onto the domain bounds while keeping the active face-defining coordinate fixed at its boundary value. For an unconstrained gradient step z originating from a boundary face along dimension j with active limit $c_j \in \{\underline{x}_j, \overline{x}_j\}$, the projection is defined element-wise for $i = 1, \ldots, n_x$ as:

$$\left[\Pi_{\partial \mathcal{X}_V}(z)\right]_i = \begin{cases} c_j, & \text{if } i = j, \\ \text{clamp}\left(z_i, \underline{x}_i, \overline{x}_i\right), & \text{if } i \neq j. \end{cases} \tag{5.32}$$

To encourage the region of attraction to expand during co-training, the estimated boundary minimum is intentionally overestimated by applying a growth factor $\chi > 1$:

$$\tilde{\rho} = \max\left(\rho_{\min}, \chi \cdot \min_{x \in \mathcal{B}_\partial} V_\phi(x)\right). \tag{5.33}$$

By targeting a value for $\tilde{\rho}$ slightly above the currently safe maximum, the formulation deliberately leads to small violations near the boundary. These violations are subsequently captured by the adversarial replay buffer, pushing the optimization to expand the sublevel set. Finally, to reduce fluctuations caused by changes in the estimated boundary minimum during training, the gating bound ρ is updated using an exponential moving average (EMA):

$$\rho \leftarrow \upsilon_\rho \rho + (1 - \upsilon)\tilde{\rho}, \tag{5.34}$$

where $\upsilon_\rho \in [0, 1)$ denotes the EMA decay rate.

5.2.5 Overall Training Procedure

The complete falsification-guided training procedure is illustrated in Fig. 5.7. It consists of an outer loop for data generation and an inner loop for network optimization. At the beginning of each outer epoch, boundary samples $\mathcal{B}_\partial$ are used to estimate the current sublevel bound ρ, and the ROA shell batch $\mathcal{B}_{\text{ROA}}$

is redrawn. Based on the updated estimation of ρ, the active LiRPA regions are selected and PGD-based falsification is performed within the corresponding sublevel set. The resulting counterexamples are added to $\mathcal{C}$, after which the explorative state pool $\mathcal{S}$ is resampled. The inner loop is executed K_{steps} times per outer epoch. At each step, a mini-batch $\mathcal{B}_\rho$ is sampled from $\mathcal{S} \cup \mathcal{C}$ and evaluated together with the active LiRPA regions, the ROA shell batch, and Sobol batches $\mathcal{B}_V$ and $\mathcal{B}_{\text{pol}}$. Their corresponding loss terms are combined into $\mathcal{L}_{\text{total}}$, which is minimized by backpropagation with respect to the trainable network parameters. Repeating this procedure over N_{outer} epochs progressively reduces Lyapunov-condition violations and enlarges the estimated certified region of attraction.

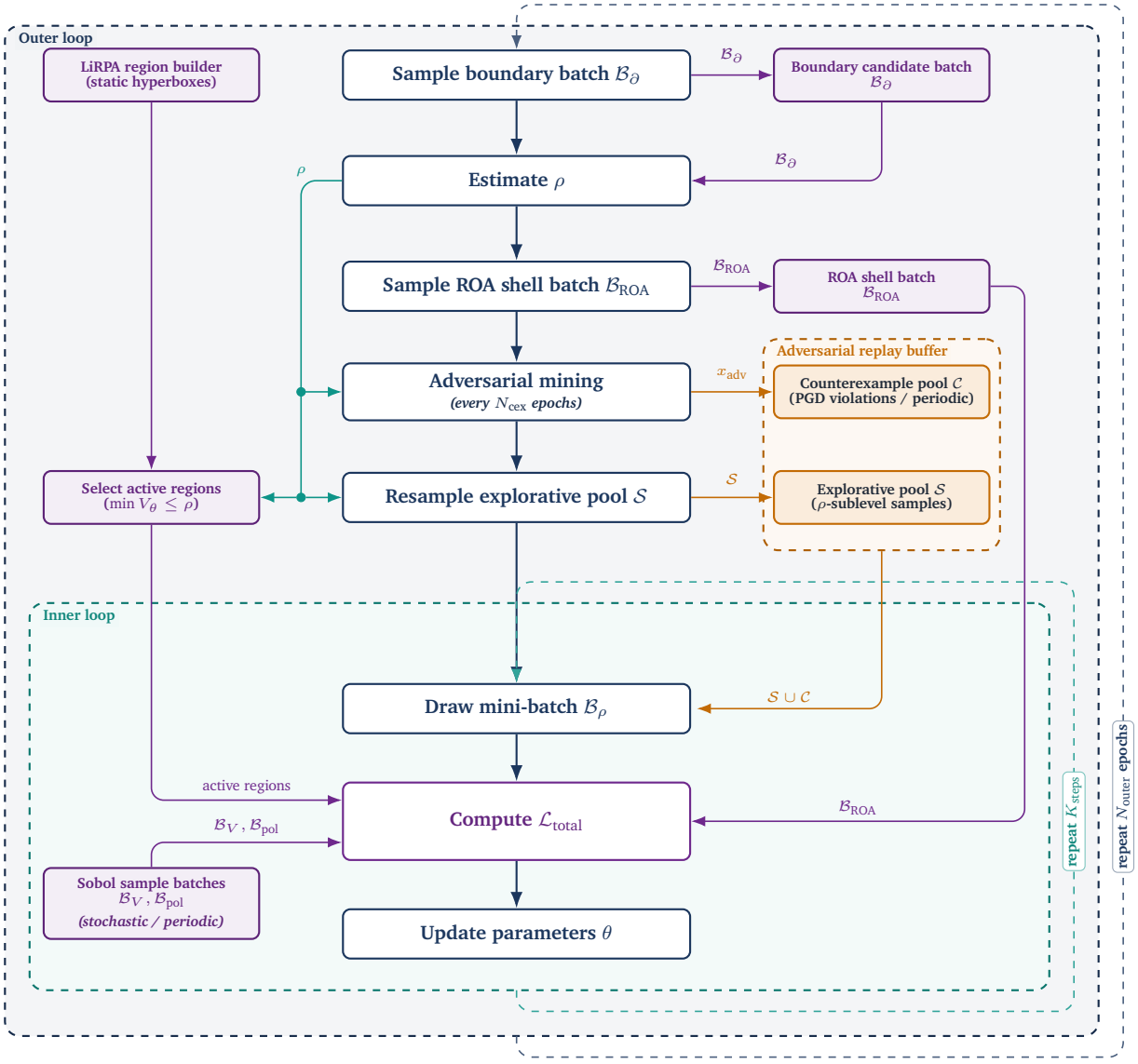

Figure 5.7: Overview of the falsification-guided training framework. The architecture is divided into an outer epoch-level loop responsible for state space exploration, boundary estimation, and adversarial mining, and an inner step-level loop that optimizes the network parameters based on the generated sample pools.

After completion of the training procedure, the optimized parameters of the policy π_V and the Lyapunov candidate V_ϕ yield a candidate control-Lyapunov pair. Although falsification-guided adversarial training

aims to eliminate counterexamples within the trained state space, it does not guarantee the complete absence of stability violations across the continuous state space. To move from empirical validation to a formal stability proof, the learned candidate pair (π_V, V_ϕ) is subsequently subjected to formal neural network verification, as detailed in the following section.

5.3 Formal Verification Methodology

This section adapts the search algorithm for the maximal certifiable sublevel set $\mathcal{V}_\rho$ proposed in [7]. Specifically, for a given target level $\rho > 0$, formal verification aims to certify that all states within the candidate region $\mathcal{V}_\rho := \{x \in \mathcal{X}_C \mid V_\phi(x) \leq \rho\}$ satisfy controlled regional invariance and exponential decay:

$$V_\phi(x) \leq \rho \implies \begin{cases} x^+ \in \mathcal{X}_C, \\ V_\phi(x^+) \leq (1-\kappa) V_\phi(x), \end{cases} \tag{5.35}$$

where $\kappa \in (0,1)$ specifies the targeted convergence rate and $x^+ = f(x, \pi_\theta(x))$ denotes the discrete one-step state transition.

Before evaluating the stability conditions, the equilibrium $x^\star$ must be explicitly excluded from the bounded certification domain $\mathcal{X}_C := \{x \in \mathbb{R}^n \mid \underline{x}_C \leq x \leq \overline{x}_C\} \subseteq \mathcal{X}_V$. By construction, $V_\phi(x^\star) = 0$ and $x^+ = x^\star$ hold at the equilibrium, causing the strict positivity and strict decrease conditions to evaluate to false at this exact point. Since these conditions are only required to hold for $x \neq x^\star$, and positive definiteness is structurally enforced by the network architecture, formal verification is instead performed over the punctured evaluation domain $\mathcal{X}_{\text{eval}} := \mathcal{X}_C \setminus \mathcal{X}_{\text{hole}}$, where $\mathcal{X}_{\text{hole}}$ denotes a small hyper-rectangular region centered at $x^\star$. For the solver to handle the punctured certification domain, $\mathcal{X}_{\text{eval}}$ is split into multiple subregions, as shown in Fig. 5.8.

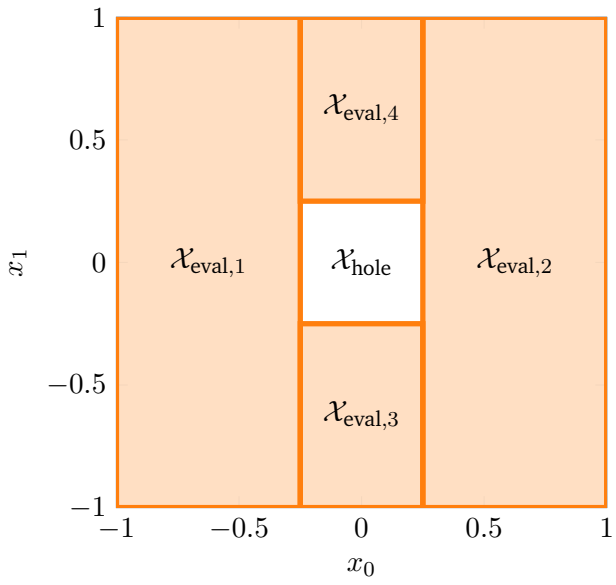

Figure 5.8: 2D Hole exclusion example with $\mathcal{X}_{\text{hole}} := \{x \in \mathbb{R}^2 \mid \|x\|_\infty < 0.25\}$.

To certify property (5.35) over $\mathcal{X}_{\text{eval}}$ using $\alpha\beta$-CROWN [16], [17], the implication is rewritten into a disjunctive formulation supported by the verifier: a state must either lie outside $\mathcal{V}_\rho$ or satisfy all stability conditions inside $\mathcal{X}_{\text{eval}}$. The individual logical terms are defined as follows:

$$\Phi_{\text{sub}} \equiv V_\phi(x) > \rho, \tag{5.36a}$$
$$\Phi_{\text{pos}} \equiv V_\phi(x) > 0, \tag{5.36b}$$
$$\Phi_{\text{inv}} \equiv \underline{x}_C \leq x^+ \leq \overline{x}_C, \tag{5.36c}$$
$$\Phi_{\text{dec}} \equiv (1-\kappa)V_\phi(x) > V_\phi(x^+). \tag{5.36d}$$

Here, (5.36a) excludes states outside the target sublevel set, while (5.36b) to (5.36d) explicitly enforce positive definiteness, state constraint invariance, and exponential decrease. Combining these terms yields the overall state-wise specification:

$$\Phi(x;\rho) \equiv \Phi_{\text{sub}} \vee \left(\Phi_{\text{pos}} \wedge \Phi_{\text{inv}} \wedge \Phi_{\text{dec}}\right). \tag{5.37}$$

To determine whether a candidate level ρ is valid across the entire continuous evaluation domain, the global verification condition $\Psi(\rho)$ evaluates whether $\Phi(x;\rho)$ holds for all states:

$$\Psi(\rho) \equiv \forall x \in \mathcal{X}_{\text{eval}} : \Phi(x;\rho). \tag{5.38}$$

The $\alpha\beta$-CROWN verifier evaluates $\Psi(\rho)$ iteratively within the scaling and bisection procedure outlined in Algorithm 2. The search operates in two distinct phases: First, a scaling phase exponentially expands or contracts ρ by a factor $q > 1$ to bracket the valid sublevel set boundary within an interval $[\rho_{\text{lo}}, \rho_{\text{up}}]$. Second, a bisection phase narrows this interval until the gap falls below a tolerance ε. Upon termination, the lower bound yields the maximum certified value $\rho^\star \approx \rho_{\text{lo}}$. Compared to the proposed formulation by Yang et al. [7], explicit iteration limits N_{scale} and N_{bisect} are integrated into both phases. This caps the maximum number of verifier calls, effectively preventing excessive computational overhead on difficult verification problems.

Algorithm 2: Certified Value Search via Scaling and Bisection

Input: $\rho_0, \rho_{\min}$, evaluator Ψ, scale $q > 1$, tol ε, limits $N_{\text{scale}}, N_{\text{bisect}}$
Output: Max certified value $\rho^\star$

$\rho \leftarrow \max(\rho_0, \rho_{\min})$;
// Phase 1: Scaling
if $\Psi(\rho)$ **then**
  for $k \leftarrow 1$ **to** N_{scale} **do**
    if $\neg\Psi(\rho)$ **then**
      break;
    end
    $\rho_{\text{lo}} \leftarrow \rho$; $\rho \leftarrow \rho \cdot q$;
  end
  $\rho_{\text{up}} \leftarrow \rho$;
else
  for $k \leftarrow 1$ **to** N_{scale} **do**
    if $\Psi(\rho) \vee \rho = \rho_{\min}$ **then**
      break;
    end
    $\rho_{\text{up}} \leftarrow \rho$; $\rho \leftarrow \max(\rho_{\min}, \rho/q)$;
  end
  if $\neg\Psi(\rho)$ **then**
    return 0 ; // No valid value $\geq \rho_{\min}$
  end
  $\rho_{\text{lo}} \leftarrow \rho$;
end
// Phase 2: Bisection
for $k \leftarrow 1$ **to** N_{bisect} **do**
  if $\rho_{\text{up}} - \rho_{\text{lo}} \leq \varepsilon$ **then**
    break;
  end
  $\rho_{\text{mid}} \leftarrow \frac{1}{2}(\rho_{\text{lo}} + \rho_{\text{up}})$;
  if $\Psi(\rho_{\text{mid}})$ **then**
    $\rho_{\text{lo}} \leftarrow \rho_{\text{mid}}$;
  else
    $\rho_{\text{up}} \leftarrow \rho_{\text{mid}}$;
  end
end
return ρ_{lo};

6 Experimental Evaluation and Certification Results

6.1 System Modeling and Problem Formulation

For our experimental evaluation, two benchmark systems are selected: the double integrator and the cartpole. While the double integrator serves as a low-dimensional linear reference baseline, the cartpole represents a nonlinear benchmark to stress-test the co-training and falsification loop. In the following, both system models and their optimization problems are formulated, followed by the shared implementation pipeline and the corresponding hyperparameter configurations.

6.1.1 Benchmark Systems

Double Integrator. The double integrator is a simple second-order linear system. Therefore, it serves as an ideal baseline system in the learning and certification pipeline. Discretizing the continuous-time dynamics using the explicit Euler method with a sampling time of $\Delta t = 0.1\,\text{s}$ leads to the discrete-time state-space equation:

$$x_{k+1} = f_{\text{DI}}(x_k, u_k) = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} x_k + \begin{bmatrix} 0 \\ \Delta t \end{bmatrix} u_k, \tag{6.1}$$

where $x_k = \begin{bmatrix} p_k & v_k \end{bmatrix}^\top$ is the state vector containing the position and velocity of the system, and u_k denotes the applied force. The state, terminal, and input bounds are chosen as:

$$\underline{x} = \underline{x}_{\text{MPC}} = \begin{bmatrix} -10 \\ -10 \end{bmatrix}, \quad \overline{x} = \overline{x}_{\text{MPC}} = \begin{bmatrix} 10 \\ 10 \end{bmatrix}, \quad \underline{x}_f = \begin{bmatrix} -5 \\ -5 \end{bmatrix}, \quad \overline{x}_f = \begin{bmatrix} 5 \\ 5 \end{bmatrix}, \quad \underline{u} = -10, \quad \overline{u} = 10.$$

For data generation, the optimal control problem in (2.4) is solved with a prediction horizon of $N = 20$ stages. States and control inputs are penalized using the weighting matrices $Q = \text{diag}(15, 1)$ and $R = 0.1$. The terminal weight matrix $P = \begin{bmatrix} 80.137 & 12.4 \\ 12.4 & 6.504 \end{bmatrix}$ is computed from the discrete algebraic Riccati equation (DARE), as described in (2.9).

Cartpole. The inverted pendulum on a cart, commonly referred to as the cartpole system, is an underactuated nonlinear system. There are four states in the system: the cart position p, the cart velocity v, the pendulum angle φ, and the pendulum angular velocity ω. Here, the discrete-time state vector is defined as $x_k = \begin{bmatrix} p_k & v_k & \varphi_k & \omega_k \end{bmatrix}^\top$, and the control input u_k is the force F_k applied to the

cart. Discretizing the continuous-time dynamics using the explicit Euler method with a sampling time of $\Delta t = 0.05\,\text{s}$ leads to the discrete-time state-space equation:

$$x_{k+1} = f_{\text{CP}}(x_k, u_k) = \begin{bmatrix} p_k + \Delta t\, v_k \\ v_k + \Delta t\, \frac{u_k + m l_p \omega_k^2 \sin(\varphi_k) - mg \sin(\varphi_k)\cos(\varphi_k)}{M + m - m\cos^2(\varphi_k)} \\ \varphi_k + \Delta t\, \omega_k \\ \omega_k + \Delta t\, \frac{-u_k \cos(\varphi_k) - m l_p \omega_k^2 \sin(\varphi_k)\cos(\varphi_k) + (M+m) g \sin(\varphi_k)}{l_p (M + m - m\cos^2(\varphi_k))} \end{bmatrix}, \tag{6.2}$$

where $M = 1\,\text{kg}$ is the mass of the cart, $m = 0.1\,\text{kg}$ is the mass of the pendulum, $l_p = 0.8\,\text{m}$ is the length of the pendulum, and $g = 9.81\,\text{m/s}^2$ is the gravitational acceleration. The corresponding state, terminal, and input bounds are defined as:

$$\underline{x} = \underline{x}_f = \begin{bmatrix} -2 \\ -10 \\ -3\pi \\ -10 \end{bmatrix}, \qquad \overline{x} = \overline{x}_f = \begin{bmatrix} 2 \\ 10 \\ 3\pi \\ 10 \end{bmatrix}, \qquad \underline{u} = -80, \qquad \overline{u} = 80.$$

Instead of sampling initial states uniformly across the entire state space $\mathcal{X}$, trajectories are initialized from a smaller sampling domain $\mathcal{X}_{\text{MPC}} := [\underline{x}_{\text{MPC}}, \overline{x}_{\text{MPC}}]$ with

$$\underline{x}_{\text{MPC}} = \begin{bmatrix} -2 \\ -10 \\ -\pi \\ -10 \end{bmatrix}, \qquad \overline{x}_{\text{MPC}} = \begin{bmatrix} 2 \\ 10 \\ \pi \\ 10 \end{bmatrix}.$$

The cartpole MPC setup uses a prediction horizon of $N = 40$ stages, with weighting matrices $Q = \text{diag}(10, 1, 10, 0.01)$ and $R = 0.5$. Furthermore, the terminal cost weight is set to

$$P = \begin{bmatrix} 2112.121 & 957.461 & -3343.435 & -882.157 \\ 957.461 & 699.028 & -2562.405 & -682.313 \\ -3343.435 & -2562.405 & 10833.55 & 2785.542 \\ -882.157 & -682.312846 & 2785.542 & 737.426 \end{bmatrix}$$

6.1.2 Experimental Pipeline and Training Setup

Expert Data Generation. The closed-loop expert trajectories are computed using the open-source framework `acados` [23], by solving the MPC optimization problem in (2.4). For both systems, a dataset $\mathcal{D}_\pi$ of 20,000 feasible trajectories is generated by uniformly sampling initial states from $\mathcal{X}_{\text{MPC}}$. To ensure distinct data points, a state-wise minimum Euclidean distance filter is enforced between initial samples: 0.001 for the double integrator, and $\begin{bmatrix} 0.01 & 0.001 & 0.001 & 0.001 \end{bmatrix}^\top$ for the cartpole. Each closed-loop system is simulated over $k = 40$ steps for the double integrator and $k = 200$ steps for the cartpole to capture full stabilization and swing-up behaviors. The detailed solver configurations are shown in **????**.

Put in actual cartpole values

Imitation Policy Training. The imitation policy π_θ is based on the architecture defined in (5.3), where an inner network h_θ is combined with state-wise clamping to ensure constraint satisfaction. An $80/20$ train-validation split is utilized to monitor generalization and drive the learning rate scheduler. Optimization is performed using Adam with a `ReduceLROnPlateau` scheduler that reduces the learning rate whenever the validation loss fails to improve for a given patience. To preserve imitation accuracy close to the origin, a state-dependent loss weighting $w_x(x) \in [w_{\min}, 1]$ is applied. Whereas only one hidden layer with 24 neurons is sufficient for the double integrator, the cartpole uses two hidden layers with 24 neurons each to be able to learn its nonlinearity. In addition, Table 6.1 lists all imitation learning hyperparameters for both systems.

Table 6.1: Imitation learning hyperparameter configuration for both benchmark systems.

Parameter	**Symbol**	**Double Integrator**	**Cartpole**
Architecture			
Activation	σ	ReLU	
Hidden layers	$L-1$	1	2
Hidden neurons	n_h	24	
Training & Optimization			
Epochs	E	1000	200
Learning rate	η	0.001	
Batch size	$\lvert\mathcal{B}_\pi\rvert$	1024	512
Dataset split	–	$0.8\,\lvert\mathcal{D}_\pi\rvert$ / $0.2\,\lvert\mathcal{D}_\pi\rvert$	
Policy domain	$\mathcal{X}_\pi$	$[-10, 10]^2$	$[-2, 2] \times [-10, 10] \times [-3\pi, 3\pi] \times [-10, 10]$
Ref. decay rate	γ	1.0	1.2
Min. state weight	$w_{\min}$	0.5	
Weight decay	λ_{ℓ_2}	0.0001	
BC loss weight	λ_{BC}	1.0	0.5
Invariance weight	λ_f	1.0	5.0
LR scheduler	–	ReduceLROnPlateau	
Scheduler params	–	factor $= 0.5$, patience $= 5$	factor $= 0.5$, patience $= 4$

Neural Lyapunov Function Co-Training. The Lyapunov candidate $V_\phi(x)$ formulated in (5.10) combines an inner network g_ϕ with a positive definite quadratic term $(\varepsilon I + R_\phi^\top R_\phi)(x - x^\star)$. The matrix R_ϕ is initialized from the discrete algebraic Riccati equation (DARE) solution matrix P of the linearized dynamics at the origin using the eigendecomposition $P = V\Lambda V^\top$ as $R_\phi = \Lambda^{1/2}V^\top$. For the double integrator, R_ϕ remains fixed throughout training, which eliminates the need for penalty terms related to the Frobenius norm and scaling. Conversely, for the cartpole, g_ϕ is parameterized with two hidden layers of 24 neurons each and R_ϕ is not fixed. To compute the LiRPA-condition loss $\mathcal{L}_{\text{L}}$, the state space $\mathcal{X}_V$ is partitioned into a uniform hyper-rectangular grid. The exclusion region $\mathcal{X}_{\text{hole}}$, further defined in Section 5.3, is used to avoid numerical issues near equilibrium. Additionally, the co-training and falsification hyperparameters are listed in Table 6.2.

Table 6.2: Hyperparameter configuration for the falsification-guided co-training.

Parameter	Symbol	Double Integrator	Cartpole
V_ϕ Architecture			
Activation	σ	ReLU	
Hidden layers	$L-1$	1	2
Hidden neurons	n_h	24	
Training & Optimization			
Outer epochs	N_{outer}	2000	
Inner steps / epoch	K_{steps}	10	
Batch size	$\lvert\mathcal{B}_\rho\rvert$	2048	
Lyapunov LR	η	0.001	
Policy LR	η_π	0.0001	
Weight decay	λ_{ℓ_2}	10^{-6}	
Lyapunov domain	$\mathcal{X}_V$	$[-10,10]^2$	$[-2,2]\times[-10,10]$ $\times[-3\pi,3\pi]\times[-10,10]$
Loss Formulation & Weights			
ρ-gated loss weight	$\lambda_{\rho-\text{gated}}$	500.0	
Invariance loss weight	λ_{inv}	1.0	
ROA loss weight	λ_{ROA}	5.0	
LiRPA loss weight	λ_{L}	15.0	
Policy reg. weight	λ_{pol}	0.1	
Decrease rate	κ	0.01	
Rel. decrease offset	ε_{rel}	0.01	
Falsification & Adversarial Replay Buffer			
PGD step size	η_{PGD}	0.001	
PGD iterations	I	20	
Max. buffer age	τ_{max}	5	
Exploration pool size	$N_{\mathcal{S}}$	8192	
Yield EMA decay	$\upsilon_{\mathcal{C}}$	0.8	
Min. CE batch fraction	r_{min}	0.2	
Max. CE batch fraction	r_{max}	0.5	
ρ-Sublevel Estimation			
Min. sublevel bound	ρ_{min}	10^{-6}	
Sublevel growth factor	χ	1.12	
Gate sharpness	s	10.0	
Resample margin	m_ρ	1.5	
EMA smoothing factor	υ_ρ	0.8	

Formal Verification Setup. To formally certify the region of attraction and compute the maximal certifiable sublevel set $\mathcal{V}_{\rho^\star}$, the verification pipeline uses the scaling and bisection search in Algorithm 2, which relies on α,β-CROWN [16], [17]. As described in Section 5.3, the verification is performed over the punctured state space $\mathcal{X}_{\text{eval}} = \mathcal{X}_C \setminus \mathcal{X}_{\text{hole}}$. All verifier settings and hyperparameter choices are listed in Table 6.3.

Table 6.3: Formal verification hyperparameter configuration for both benchmark systems.

Parameter	Symbol	Double Integrator	Cartpole
Domain & Stability Specification			
Decrease rate	κ	0.01	
Verification domain	$\mathcal{X}_C$	$[-10, 10]^2$	$[-2, 2] \times [-10, 10] \times [-3\pi, 3\pi] \times [-10, 10]$
Origin exclusion	$\mathcal{X}_{\text{hole}}$	$[-0.05, 0.05] \times [-0.15, 0.15]$	$[-0.01, 0.01] \times [-0.001, 0.001]^3$
Bounding method	–	CROWN	
Scaling & Bisection Search			
Min. sublevel bound	$\rho_{\min}$	10^{-6}	
Scale factor	q	1.2	
Max. scaling steps	N_{scale}	20	
Max. bisection steps	N_{bisect}	40	
Bisection tolerance	$\varepsilon_{\text{bisect}}$	10^{-3}	
α,β-CROWN Verifier Settings			
Batch size	$\lvert\mathcal{B}_{\text{ver}}\rvert$	32	
Input partitions	p_{split}	2	
Verifier tolerance	$\varepsilon_{\text{cond}}$	10^{-6}	

6.2 Linear System Results: Double Integrator

Since the double integrator is a simple linear system, it serves as a good baseline to determine the performance of the proposed co-training and falsification pipeline. Looking at the defined optimal value function of the double integrator MPC in Fig. 6.1, one can see that there exists a stable region around the origin. However, this region is only estimated based on the datasets' feasibility and lyapunov decrease using the $V = x^\top P x$ as a Lyapunov function. The dataset provides a practical foundation to learn a policy that is able to approximate the optimal control performance, while preserving the system's stability. To show that the learned and converged controller is able to approximate the expert's performance, the error band between the closed-loop learned- and expert controller is shown in Fig. 6.2. The error band is computed using the same initial states x_0 drawn from the validation dataset for both controllers. Then, the element-wise difference of the closed-loop trajectories is calculated. Note that the states of the learned policys deviate by less than 5% from the experts policy, while the inputs deviate by up to 70%. The initial loss $\mathcal{L}_\pi$ for this particular policy starts with 23.787, decreases to 0.404 in the first 50 epochs and then converges to 0.047 after 1000 epochs, which can be seen in Fig. A.1.

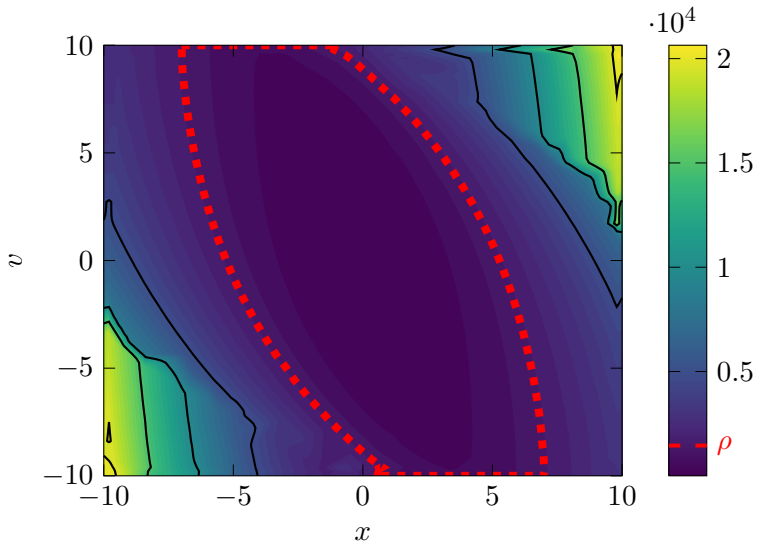

Figure 6.1: Double integrator Lyapunov landscape of the MPC's optimal value function with an estimated ROA: $1.45 \cdot 10^3$.

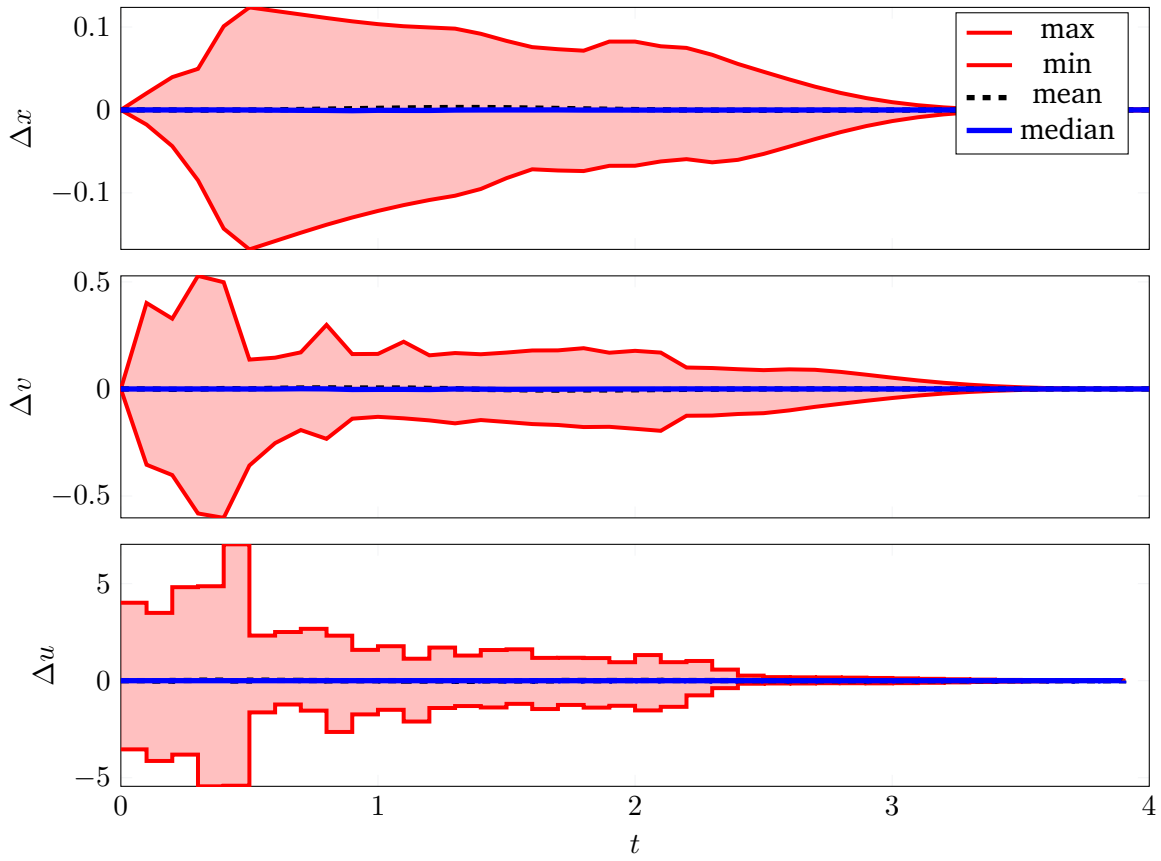

Figure 6.2: Double integrator error band. $x_\theta - x_{\text{MPC}}$

With the learned imitation policy, the Lyapunov candidate can be learned in the co-training framework. Here, the loss $\mathcal{L}_{\text{total}}$ converges from 77.843 to 1.37, while the estimated $\tilde{\rho}$ also converges from 57.082 to 613.4. Based on the boundary samples used for the sublevel set estimation $\mathcal{B}_\partial$, the estimated feature term share $|g_\phi(x) - g_\phi(x^\star)|$ is $17.7\,\%$ and the positive-definite auxiliary term share $\left\|(\varepsilon I + R_\phi^\top R_\phi)(x - x^\star)\right\|_1$ is about $82.3\,\%$. This indicates that while the auxiliary term accounts for most of the magnitude and guarantees strict positive definiteness, the neural component provides the necessary local deformations to satisfy the decrease conditions.

After training the candidate $V_\phi(x)$, the final pipeline step is to find a formally verified sublevel set $\mathcal{V}_\rho$ to ensure stability of the closed-loop dynamics. Given the setup in Table 6.3, the verifier finds the largest formally verified $\rho^\star = 665.66$, as shown in Fig. 6.3. In comparison to the empirical ROA of the MPC's optimal value function, visualized in Fig. 6.1, the certified ROA of the learned policy covers an area of 260.27, which is $42.1\,\%$ larger than the MPC baseline with an area of 183.22. The volume measurements for both regions were computed using Monte-Carlo integration over the state space $\mathcal{X}$.

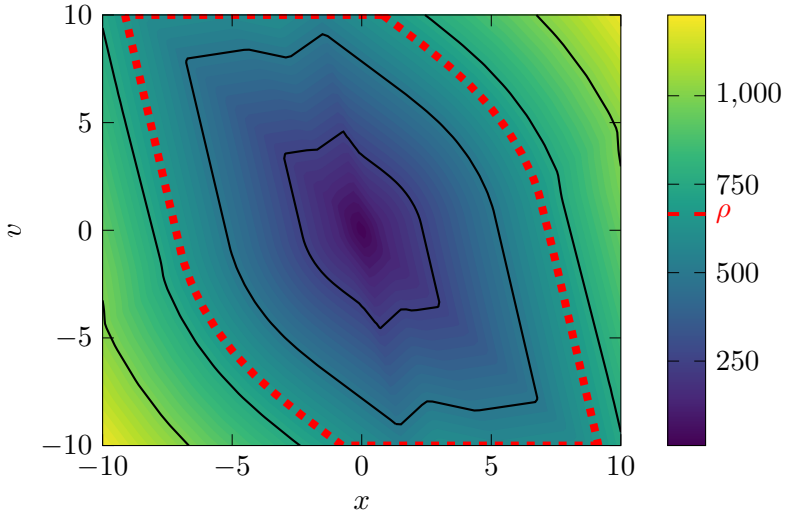

Figure 6.3: Double integrator Lyapunov landscape and ROA: $\rho^\star = 665.66$

6.3 Nonlinear System Results: Cartpole

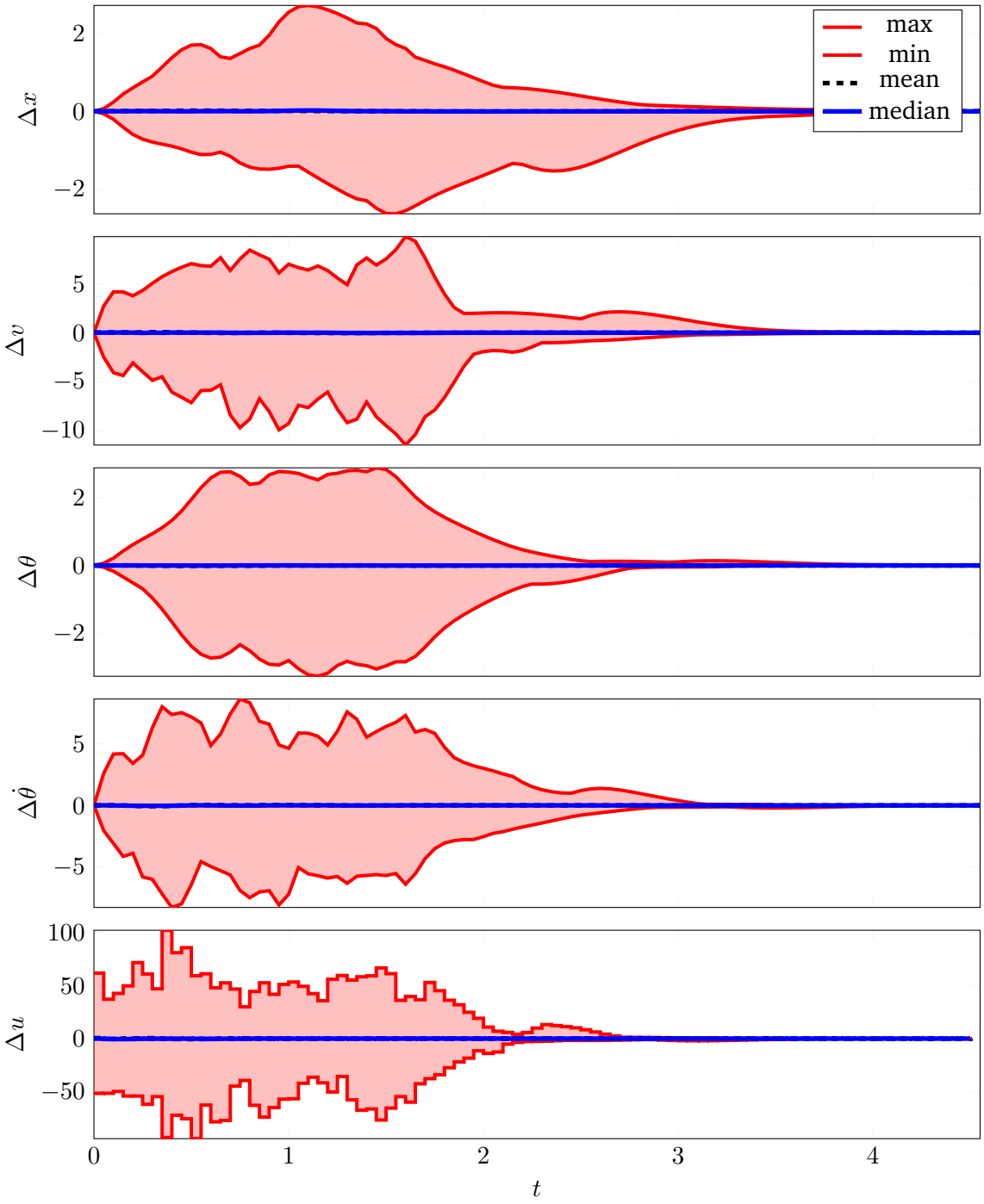

Figure 6.4: Cartpole error band. $x_\theta - x_{\text{MPC}}$

7 Discussion

7.1 Qualitative Discussion of Results

7.2 Challenges with Nonlinear Dynamics

7.3 Scalability and Limitations

8 Conclusion

A Appendix

A.0.1 Imitation Learning

Note: If the MPC problem becomes infeasible at any time during one trajectory, all training pairs associated with that trajectory are discarded.

To reduce the overrepresentation of regions with high sampling rates, near-duplicate state-action pairs are removed during preprocessing. The concatenated state-action vectors are normalized and mapped onto an adaptive voxel grid. For each occupied voxel, at most one representative pair is retained.

A.0.2 α,β-CROWN Verification Backend Configuration

To ensure reproducibility of the formal certification pipeline, the default configuration for the α,β-CROWN verification backend is summarized in Table A.1. The backend utilizes input-space Branch-and-Bound (`input_bab`) combined with CROWN linear bound propagation and Split-by-Score (`sb`) branching.

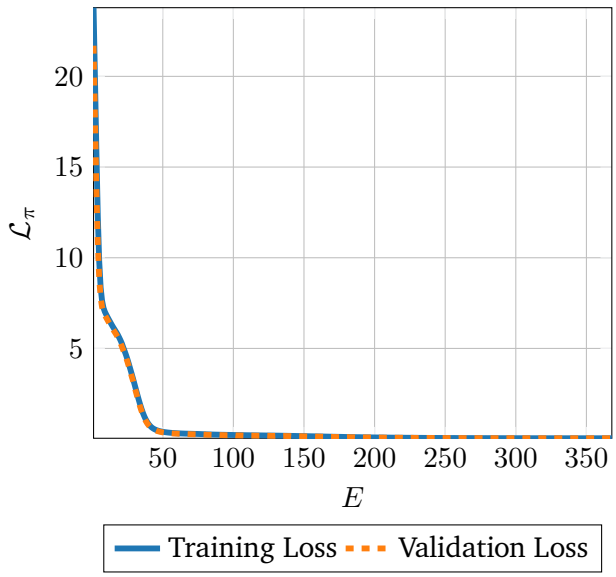

Figure A.1: Double integrator imitation learning policy training and validation loss over epochs.

Table A.1: Detailed configuration parameters for the α,β-CROWN verification backend.

Parameter	Configuration Key
General Strategy	
Verifier mode	`general.complete_verifier`
Incomplete verification pre-pass	`general.enable_incomplete_verification`
Bounding & Solver	
Bound propagation method	`solver.bound_prop_method`
Batch size (subdomains per GPU step)	`solver.batch_size`
Branch-and-Bound (BaB)	
Branching heuristic	`bab.branching.method`
Input space splitting	`bab.branching.input_split.enable`
Split partitions per dimension	`bab.branching.input_split.split_partitions`
IBP intermediate enhancement	`bab.branching.input_split.ibp_enhancement`
Monotonic bound comparison	`bab.branching.input_split.compare_with_old_bounds`
Adversarial check in tree	`bab.branching.input_split.adv_check`
Adversarial Falsification	
PGD attack scheduling	`attack.pgd_order`
Termination & Tolerances	
Decision threshold	`bab.decision_thresh`
Domain timeout limit	`bab.timeout`
Maximal active domains	`bab.max_domains`

Bibliography

[1] L. Grüne and J. Pannek, *Nonlinear Model Predictive Control: Theory and Algorithms* (Communications and Control Engineering), 2nd ed. Cham: Springer, 2017, 456 pp., ISBN: 978-3-319-46024-6.

[2] J. B. Rawlings, D. Q. Mayne, and M. Diehl, *Model Predictive Control: Theory, Computation and Design*, 2nd ed. Santa Barbara: Nob Hill Publishing, LLC, 2020, 623 pp., ISBN: 978-0-9759377-5-4.

[3] B. Karg and S. Lucia, "Efficient Representation and Approximation of Model Predictive Control Laws via Deep Learning," *IEEE Transactions on Cybernetics*, vol. 50, pp. 3866–3878, 2020, ISSN: 2168-2275.

[4] M. Hertneck, J. Köhler, S. Trimpe, and F. Allgöwer, "Learning an Approximate Model Predictive Controller with Guarantees," *IEEE Control Systems Letters*, vol. 2, pp. 543–548, 2018, ISSN: 2475-1456.

[5] L. Hewing, K. P. Wabersich, M. Menner, and M. N. Zeilinger, "Learning-Based Model Predictive Control: Toward Safe Learning in Control," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 3, pp. 269–296, 2020, ISSN: 2573-5144.

[6] R. Reiter, J. Hoffmann, D. Reinhardt, *et al.*, "Synthesis of model predictive control and reinforcement learning: Survey and classification," *Annual Reviews in Control*, vol. 61, p. 101 045, 2026, ISSN: 1367-5788.

[7] L. Yang, H. Dai, Z. Shi, C.-J. Hsieh, R. Tedrake, and H. Zhang, "Lyapunov-stable Neural Control for State and Output Feedback: A Novel Formulation," in *International Conference on Machine Learning*, Vienna, Austria, 2024, pp. 56 033–56 046.

[8] C. C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*, 2nd ed. Cham: Springer International Publishing AG, 2023, 529 pp., ISBN: 978-3-031-29642-0.

[9] H. Zhang, T.-W. Weng, P.-Y. Chen, C.-J. Hsieh, and L. Daniel, "Efficient Neural Network Robustness Certification with General Activation Functions," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[10] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and Harnessing Adversarial Examples," in *International Conference on Learning Representations*, San Diego, CA, USA, 2015.

[11] C. Szegedy, W. Zaremba, I. Sutskever, *et al.*, "Intriguing properties of neural networks," in *International Conference on Learning Representations*, Banff, AB, Canada, 2014.

[12] European Parliament and Council of the European Union, *Regulation (EU) 2024/1689 of 13 June 2024 Laying down Harmonised Rules on Artificial Intelligence (Artificial Intelligence Act)*, 2024.

[13] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer, "Reluplex: An efficient SMT solver for verifying deep neural networks," in *Computer Aided Verification*, R. Majumdar and V. Kunčak, Eds., Cham: Springer International Publishing, 2017, pp. 97–117, ISBN: 978-3-319-63387-9.

[14] V. Tjeng, K. Y. Xiao, and R. Tedrake, "Evaluating Robustness of Neural Networks with Mixed Integer Programming," in *International Conference on Learning Representations*, New Orleans, LA, USA, 2019.

[15] R. Bunel, I. Turkaslan, P. H. S. Torr, P. Kohli, and M. P. Kumar, "A Unified View of Piecewise Linear Neural Network Verification," in *Advances in Neural Information Processing Systems*, vol. 31, 2018, pp. 4795–4804.

[16] K. Xu, H. Zhang, S. Wang, *et al.*, "Fast and Complete: Enabling Complete Neural Network Verification with Rapid and Massively Parallel Incomplete Verifiers," in *International Conference on Learning Representations*, Austria, 2021.

[17] S. Wang, H. Zhang, K. Xu, *et al.*, "Beta-CROWN: Efficient bound propagation with per-neuron split constraints for complete and incomplete neural network verification," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 29 909–29 921.

[18] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. Chichester, UK: Wiley, 2016, ISBN: 978-1-119-12153-4.

[19] S. Kollmannsberger, D. D'Angella, M. Jokeit, and L. Herrmann, *Deep Learning in Computational Mechanics: An Introductory Course* (Studies in Computational Intelligence 977). Cham: Springer, 2021, 104 pp., ISBN: 978-3-030-76587-3.

[20] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *International Conference on Learning Representations*, San Diego, CA, USA, 2015.

[21] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards Deep Learning Models Resistant to Adversarial Attacks," in *International Conference on Learning Representations*, Vancouver, Canada, 2018.

[22] S. Joe and F. Y. Kuo, "Constructing Sobol Sequences with Better Two-Dimensional Projections," *SIAM Journal on Scientific Computing*, vol. 30, no. 5, pp. 2635–2654, Jan. 2008, ISSN: 1064-8275, 1095-7197.

[23] R. Verschueren, G. Frison, D. Kouzoupis, *et al.*, "Acados—a modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation*, vol. 14, pp. 147–183, 2022, ISSN: 1867-2957.